

Optimization for Prescriptive Analytics Notes

Michelle L. Wu

Summer 2024 - Mini 1

Optimization for Prescriptive Analytics

Taught by: Dr. Willem-Jan van Hove

Contents

1	Background	5
1.1	What is Optimization?	5
1.2	The Role of Optimization in Analytics	5
1.3	Types of Optimization Models	5
1.4	The Optimization Process	5
1.5	Optimization Software and Tools	6
1.6	Real-World Applications	6
2	Linear Programming	7
2.1	Theory & Terminology	7
2.2	Diet Problem	8
2.3	Live Session Applications	9
2.3.1	Cash Flow Matching Problem	9
2.3.2	MLB Scheduling Problem	10
3	Linear Programming Applications	12
3.1	Transportation Problem: Production Planning	12
3.2	Mixing and Blending	13
3.3	Multi-Period Models	14
3.4	Live Session: Cash Flow Matching and Workforce Optimization	16
4	Network Flow Models	19
4.1	Theory & Terminology	19
4.2	Transshipment Problem	20
4.3	Shortest Path Problem	21
4.4	Maximum Flow Problem	22
4.5	Multi-Commodity Flow Problem	23
4.6	Live Session: LNG Tanker Scheduling	24
5	Integer Programming	27
5.1	Methodology	27
5.2	Set Covering Problem	28
5.3	Knapsack Problem and Logical Rules	29
5.4	Fixed Cost Problem	30
5.5	Live Session: Combinatorial Auction – Landing Slot Allocation	31
6	Integer Programming Applications	33
6.1	Minimum Order Problem	33
6.2	Facility Location Problem	34
6.3	Location and Distribution	35
6.4	Vehicle Routing	36
6.5	Live Session: Advanced Vehicle Routing	37
6.5.1	Overview	37

6.5.2	Network-Based IP Model	38
6.5.3	Scalability and Solver Settings	39
6.5.4	Real-World Applications	39
6.5.5	Implementation Notes	39
7	Nonlinear Programming	40
7.1	Theory & Terminology	40
7.1.1	Motivation and Scope	40
7.1.2	Key Definitions and Concepts	40
7.1.3	First and Second Order Conditions in NLP	41
7.1.4	Problem Setup	41
7.1.4	First-Order Necessary Conditions (FONC)	41
7.1.5	Second-Order Necessary Conditions (SONC)	41
7.1.6	Second-Order Sufficient Conditions (SOSC)	42
7.1.7	Constraint Qualifications (CQs)	42
7.1.8	Interpretation and Intuition	42
7.1.8	Best Practices	42
7.1.9	Duality Theory in Nonlinear Programming	42
7.1.10	Economic Interpretation of Lagrange Multipliers	43
7.1.11	Representative Applications of NLP	44
7.1.12	Solvers for NLP	44
7.1.13	Summary and Best Practices	44
7.1.14	Convexity in Nonlinear Programming	44
7.1.15	Mathematical Foundations	45
7.1.16	Convex NLP Problem Form	45
7.1.17	Key Properties and Implications	45
7.1.18	Examples of Convex NLPs	45
7.1.19	Nonconvex NLPs: Contrast and Challenge	46
7.1.20	Testing Convexity in Practice	46
7.1.21	Modern NLP Solver Implications	46
7.1.22	Conclusion and Best Practices	46
7.1.23	Selected Problem Types	46
7.1.24	Theory and Computation	47
7.1.25	Solver Landscape	47
7.1.26	Software and Tools	47
7.1.27	Modeling Tips	48
7.1.28	Common Pitfalls	48
7.1.29	Gradient-Based Algorithmic Strategies	48
7.2	Single Variable NLP	49
7.3	Multivariate NLP	50
7.4	Quadratic Programming	51
7.5	General Integer NLP (MINLP)	52
7.6	Barrier and Interior-Point Methods for Nonlinear Programming	53
7.6.1	Theoretical Properties	54
7.6.2	Barrier Method Formulation	54
7.6.3	Barrier Properties	54
7.6.4	Central Path and Barrier KKT Conditions	55
7.6.5	Barrier KKT Conditions	55
7.6.6	Primal-Dual Interior-Point Formulation	55
7.6.7	Residual Vector Definition	55
7.6.8	Primal-Dual Newton Step	56

7.6.7	Algorithmic Steps	56
7.6.8	Convergence Properties	56
7.6.9	Comparison with Active Set Methods	56
7.6.10	Applications and Solver Use	56
7.6.11	Connection to Convex Analysis	57
7.7	Sensitivity Analysis in Nonlinear Programming	57
7.7.1	Setup and Notation	57
7.7.2	Total Derivative of the Optimal Value	57
7.7.3	Differentiation of KKT Conditions	57
7.7.4	Shadow Prices and Reduced Costs in NLP	58
7.7.5	Sensitivity of the Active Set	58
7.7.6	Applications and Use Cases	58
7.7.7	Best Practices	58
7.8	Advanced Newton and Quasi-Newton Methods in NLP	58
7.8.1	Unconstrained Newton's Method	59
7.8.2	Local Convergence Theory	59
7.8.3	Equality-Constrained Newton-KKT Method	59
7.8.4	Quasi-Newton Methods: BFGS and L-BFGS	59
7.8.5	Trust-Region Newton Methods (TRON)	60
7.8.6	Globalization Strategies	60
7.8.7	Implementation in Solvers	60
7.8.8	Best Practices	61
7.9	Penalty and Augmented Lagrangian Methods for Nonlinear Programming	61
7.9.1	Quadratic Penalty Method	61
7.9.2	Exact Penalty Methods (L1 Penalty)	62
7.9.3	Augmented Lagrangian Method (ALM)	62
7.9.4	Handling Inequality Constraints	62
7.9.5	Connections to Dual Decomposition	63
7.9.6	Implementation Challenges and Solutions	63
7.9.7	Comparative View with Barrier Methods	63
7.9.8	Modern Software and Applications	63
7.10	Slater's Condition and Constraint Qualification	63
7.10.1	KKT Conditions and Dual Feasibility	64
7.10.2	Duality Gap and Sensitivity	64
7.10.3	Example: Nonlinear Dual Derivation	64
7.10.4	Extended Use Cases: Conic and Semidefinite Programming	65
7.11	Sequential Convex Programming (SCP) and Successive Convex Approximation (SCA)	65
7.11.1	Problem Structure and Reformulation	65
7.11.2	Convexification Techniques	65
7.11.3	Algorithmic Framework	66
7.11.4	Convergence Properties	66
7.11.5	Implementation Details	66
7.11.6	Mathematical Example: Cardinality-Constrained Portfolio Optimization	67
7.11.7	Applications	67

7.11.8	Software Support	67
7.12	Global Optimization for Nonconvex NLP	67
7.12.1	Branch-and-Bound for Nonconvex NLPs	68
7.12.1.1	Mathematical Framework	68
7.12.1.2	Branching and Feasibility	68
7.12.1.3	Example: Nonconvex Quadratic Program	68
7.12.2	Convex Relaxations and Underestimation	68
7.12.2.1	McCormick Envelopes	69
7.12.2.2	Factorable Function Relaxations	69
7.12.2.3	Piecewise Convex Relaxations	69
7.12.3	Deterministic vs. Stochastic Global Solvers	69
7.12.3.1	Deterministic Solvers	69
7.12.3.2	Stochastic and Metaheuristic Methods	69
7.12.4	Computational Trade-offs and Enhancements	69

1 Background

1.1 What is Optimization?

Optimization is the process of making the best possible decision given a set of constraints and objectives. In mathematical terms, it involves selecting values for decision variables that maximize or minimize an objective function, subject to a set of constraints.

Optimization models serve as abstractions of real-world decision problems. These models allow organizations to make data-driven, implementable decisions in areas such as logistics, scheduling, portfolio management, and supply chain operations.

1.2 The Role of Optimization in Analytics

In the modern analytics landscape, optimization forms the backbone of prescriptive analytics. While descriptive analytics summarizes historical data and predictive analytics forecasts future events, prescriptive analytics recommends specific decisions. Optimization directly enables this decision-making.

Prescriptive analytics: What should we do?

- Builds mathematical models of decisions.
- Finds best actions using solvers (e.g., Gurobi).
- Accounts for constraints, objectives, trade-offs.

1.3 Types of Optimization Models

Optimization models vary based on the form of their decision variables, objective functions, and constraints. Key types include:

- **Linear Programming (LP):** Continuous decision variables, linear objective, linear constraints.
- **Integer Programming (IP):** Discrete decision variables, often binary or whole numbers.
- **Nonlinear Programming (NLP):** Nonlinear relationships in objective or constraints.
- **Stochastic Programming (SP):** Incorporates uncertainty in parameters.

Each type provides powerful modeling capabilities for different kinds of problems.

1.4 The Optimization Process

To build and solve an optimization model, follow this structured process:

1. **Understand the problem:** Identify key decisions, objectives, and constraints.
2. **Define decision variables:** Represent each unknown quantity to be determined.

3. **Formulate the objective function:** A function to maximize or minimize.
4. **Write constraints:** Mathematical expressions defining limits and relationships.
5. **Implement in software:** Use tools like Python and Gurobi to encode and solve the model.
6. **Interpret the results:** Translate optimal variable values into actionable decisions.

1.5 Optimization Software and Tools

In this course, we use:

- **Gurobi:** A state-of-the-art solver for linear, integer, and nonlinear programs.
- **Gurobipy:** Python interface to Gurobi.
- **Google Colab / Jupyter Notebooks:** Interactive environments for modeling.

Students can install Gurobi locally or access it via web-based notebooks. A free academic license is available and strongly recommended for hands-on exercises.

1.6 Real-World Applications

Optimization is everywhere:

- Logistics and supply chain management
- Airline scheduling and crew assignments
- Investment portfolio optimization
- Marketing campaign planning
- Cancer treatment and radiotherapy planning

Each application domain brings unique constraints and objectives, but all benefit from the unifying language and tools of optimization.

2 Linear Programming

2.1 Theory & Terminology

Linear Programming (LP) refers to the class of optimization problems where the objective function and all constraints are linear in the decision variables.

Feasible Set

The set of all points that satisfy the constraints of a linear program is called the **feasible set**. A point in the feasible set is a **feasible solution**.

Optimal Solution

A **solution** that yields the best (maximum or minimum) value of the objective function among all feasible solutions. According to the Fundamental Theorem of Linear Programming, if an optimal solution exists, it occurs at an *extreme point* (or corner point) of the feasible region.

Objective Level Curves

These are lines (in 2D) or hyperplanes (in higher dimensions) representing equal values of the objective function. The graphical method involves "sliding" these curves outward (for maximization) or inward (for minimization) until they just touch the feasible region.

Binding Constraints

A constraint is **binding** if it holds with equality at the optimal solution; this means it directly influences the optimal value. Binding constraints define the boundary of the feasible region that the solution lies on.

Non-binding Constraints

A constraint is **non-binding** if there is slack (i.e., it is not tight) at the optimal solution. These constraints do not actively restrict the final decision.

Special Cases

- **Unique Optimal Solution:** A single best feasible point.
- **Alternative Optimal Solutions:** Multiple extreme points yield the same optimal objective value; often occurs when the objective function is parallel to a constraint boundary.
- **Unbounded Solution:** Objective value can be improved without bound within the feasible region. This typically indicates missing constraints.

- **Infeasible Problem:** No solution satisfies all constraints. This can result from conflicting constraints or overly strict requirements.

Linear programming provides the foundation for many complex decision problems and can be solved graphically for small problems (2 variables) or using industrial solvers such as Gurobi for large-scale problems.

2.2 Diet Problem

The **Diet Problem** is a classical LP problem where the goal is to choose quantities of different foods to meet nutritional requirements at minimum cost.

Problem Framing

Originally formulated in the 1940s and used by the U.S. military, the diet problem remains relevant today in school lunch design, personalized meal planning, and as a prototype for mixing problems in finance and manufacturing.

Variables

- x_b = servings of beef
- x_c = servings of chicken
- x_r = servings of rice
- x_s = servings of spaghetti

Objective

Minimize cost:

$$\min 3.2x_b + 2.4x_c + 2x_r + 2.5x_s$$

Constraints

Meet weekly nutritional requirements (at least 700 units of A, B, and C):

$$60x_b + 80x_c + 25x_r + 60x_s \geq 700 \quad (\text{Nutrient A})$$

$$20x_b + 0x_c + 50x_r + 20x_s \geq 700 \quad (\text{Nutrient B})$$

$$40x_b + 20x_c + 25x_r + 15x_s \geq 700 \quad (\text{Nutrient C})$$

Non-negativity

$$x_b, x_c, x_r, x_s \geq 0$$

Generalized Formulation

Let F be the set of food items and N the set of nutrients:

$$\begin{aligned} \min \quad & \sum_{i \in F} c_i x_i \\ \text{s.t.} \quad & \sum_{i \in F} a_{ij} x_i \geq R_j \quad \forall j \in N \\ & x_i \geq 0 \quad \forall i \in F \end{aligned}$$

Python Implementation Notes

The problem is often implemented using Gurobi with Python:

- Sets F and N are implemented as lists.
- Parameters like c_i and a_{ij} are stored in dictionaries.
- The model is built using Gurobi's `addVar`, `addConstr`, and `setObjective` functions.

Scalability

This model easily scales to real-world settings. For instance, `diet_large.ipynb` includes dozens of food items and nutrient requirements, yet follows the same LP structure.

The diet problem elegantly illustrates how LP can be used to solve real-world decisions with multiple interacting constraints and objectives.

2.3 Live Session Applications

2.3.1 Cash Flow Matching Problem

The **Cash Flow Matching Problem** is a linear programming application in financial planning and portfolio optimization. The objective is to invest in bonds such that future liabilities are met exactly or within some tolerance.

Problem Setup Given:

- A set B of available bonds
- A set T of time periods (e.g., years or quarters)
- c_b : cost of purchasing one unit of bond $b \in B$
- r_{bt} : return (cash flow) from bond b in period t
- d_t : liability (payment) due in period t

Decision Variables

- x_b : number of units to purchase of bond b

Formulation

$$\begin{aligned}
 \min \quad & \sum_{b \in B} c_b x_b \\
 \text{s.t.} \quad & \sum_{b \in B} r_{bt} x_b \geq d_t \quad \forall t \in T && \text{(Meet all liabilities)} \\
 & x_b \geq 0 \quad \forall b \in B
 \end{aligned}$$

Implementation Notes

- Bond data stored in a matrix: `return[b][t]`
- Constraints are generated over time periods T
- Can be extended to include surplus minimization or exact matching

2.3.2 MLB Scheduling Problem

The **MLB Scheduling Problem** is framed as an assignment problem to pair rival teams from the American League (AL) and National League (NL) such that the total distance traveled is minimized.

Problem Setup Given:

- A : set of American League teams
- N : set of National League teams
- d_{ij} : distance between AL team $i \in A$ and NL team $j \in N$

Decision Variables

- $x_{ij} = 1$ if team $i \in A$ is matched to team $j \in N$, else 0

Formulation

$$\begin{aligned}
 \min \quad & \sum_{i \in A} \sum_{j \in N} d_{ij} x_{ij} \\
 \text{s.t.} \quad & \sum_{j \in N} x_{ij} = 1 \quad \forall i \in A && \text{(Each AL team is matched)} \\
 & \sum_{i \in A} x_{ij} = 1 \quad \forall j \in N && \text{(Each NL team is matched)} \\
 & x_{ij} \in \{0, 1\} \quad \forall i \in A, j \in N
 \end{aligned}$$

Notes

- Input data from CSV: `MLB_distance_matrix.csv` and `MLB_teams.csv`
- Total number of pairs equals $|A| = |N|$
- Objective reflects real-world considerations: rivalry proximity and travel minimization

These two problems show how LP and its integer variants scale to financial and logistical applications. They leverage the same formulation logic but adapt to domain-specific constraints and data.

3 Linear Programming Applications

3.1 Transportation Problem: Production Planning

This model is a non-obvious extension of the transportation problem, applied to production planning over time. The objective is to decide how many units to produce for each customer in each week to maximize revenue while respecting production limits and meeting customer demand.

Theoretical Framing

This is structurally a transportation problem where:

- Sources are **weeks** (indexed by production capacity)
- Destinations are **clients** (indexed by demand)
- Arcs represent units produced in a week and assigned to a client, each with a **revenue per unit** instead of a cost.

Unlike classical transportation problems, this is a *revenue maximization* setting, requiring demand satisfaction over a time horizon.

Use Cases

- Steel and manufacturing delivery contracts
- Energy generation and delivery scheduling
- Inventory-augmented production allocation

Problem Setup

- W : set of weeks
- C : set of clients
- P_{wc} : unit price client c will pay for delivery in week w
- S_w : maximum production capacity in week w
- D_c : demand of client c over the planning horizon

Decision Variables

- x_{wc} : number of units produced in week w for client c

Formulation

$$\begin{aligned}
 \max \quad & \sum_{w \in W} \sum_{c \in C} P_{wc} x_{wc} \\
 \text{s.t.} \quad & \sum_{w \in W} x_{wc} = D_c \quad \forall c \in C && \text{(Meet all demand)} \\
 & \sum_{c \in C} x_{wc} \leq S_w \quad \forall w \in W && \text{(Weekly capacity)} \\
 & x_{wc} \geq 0 \quad \forall w \in W, c \in C
 \end{aligned}$$

Notes on Integrality

Like classical transportation models, this formulation has the integer-solution property if P_{wc} , D_c , and S_w are integers. However, integer enforcement is unnecessary in practice due to continuous prices.

Implementation Notes

- Set definitions: `Weeks`, `Clients`
- Parameters: `Price[(w,c)]`, `Demand[c]`, `Supply[w]`
- Variable: `x[(w,c)]`

3.2 Mixing and Blending

This class of problems involves combining multiple base components to form a product that meets specified properties. The distinction lies in whether the total amount is known (mixing) or unknown (blending).

Theory and Comparison

- **Mixing Problems:** Total mix is known in advance (e.g., 10 million USD invested). Weighted average constraints use fixed denominators, so no linearization needed.
- **Blending Problems:** Total mix is unknown, so ratio constraints become nonlinear. These require linearization.

Example: Portfolio Design (Mixing)

- B : set of bonds
- t_i : type of bond i (e.g., Muni, Agency, Govn)
- q_i : quality rating (lower is better)
- m_i : maturity

- p_i : yield (return) per million invested

Decision Variables: x_i : millions to invest in bond i

Objective:

$$\max \sum_{i \in B} p_i x_i$$

Constraints:

$$\begin{aligned} \sum_{i \in B} x_i &= 10 && \text{(Total investment)} \\ \sum_{i \in B: t_i = \text{Muni}} x_i &\leq 3 && \text{(Municipal cap)} \\ \sum_{i \in B} q_i x_i &\leq 1.4 \cdot \sum_{i \in B} x_i && \text{(Quality avg)} \\ \sum_{i \in B} m_i x_i &\leq 5 \cdot \sum_{i \in B} x_i && \text{(Maturity avg)} \\ x_i &\geq 0 \quad \forall i \in B \end{aligned}$$

Linearization

To remove nonlinear terms, rearrange:

$$\begin{aligned} \sum_{i \in B} (q_i - 1.4) x_i &\leq 0 \\ \sum_{i \in B} (m_i - 5) x_i &\leq 0 \end{aligned}$$

Modeling Insights

- Weighted averages are made linear by multiplying both sides of inequality.
- Logical constraints (e.g., "muni cap") are applied using conditional sums (subset sums).

Implementation Notes

- Parameters: `Yield[i]`, `Maturity[i]`, `Quality[i]`, `Type[i]`
- Variables: `x[i]`
- Dataset: `portfolio.xlsx` (see `portfolio.ipynb`)

3.3 Multi-Period Models

Multi-period linear programming models involve planning over a time horizon where decisions in one period affect the feasibility or cost in future periods. These models are commonly applied in workforce scheduling, inventory planning, and financial forecasting.

Workforce Planning Problem

A classic example is cyclic scheduling where each worker follows a fixed shift pattern (e.g., 5 days on, 2 days off), and the goal is to minimize the total workforce required while covering demand each day.

Sets and Parameters:

- W : set of weekdays (Mon through Sun)
- S : set of cyclic shifts (7 total, one starting each day)
- R_d : minimum number of workers required on day $d \in W$
- $a_{id} \in \{0, 1\}$: 1 if shift i covers day d , 0 otherwise

Decision Variables:

- x_i : number of workers assigned to shift $i \in S$

Objective:

$$\min \sum_{i \in S} x_i \quad (\text{minimize total workers})$$

Constraints:

$$\begin{aligned} \sum_{i \in S} a_{id} x_i &\geq R_d \quad \forall d \in W && (\text{cover daily demand}) \\ x_i &\geq 0 \quad \forall i \in S \end{aligned}$$

Python Implementation:

- `Demand[d]` for each day d
- `DaysInShift[i]` as a list or indicator matrix
- `x[i]` variables represent the number of workers per shift

Inventory Planning Problem

This model addresses the trade-off between production cost and inventory holding cost over multiple periods, subject to production capacity and storage constraints.

Sets and Parameters:

- M : set of months
- D_i : demand in month $i \in M$
- C_i : production cost per unit in month i
- K : production capacity per month
- H : inventory holding cost per unit per month

- S : storage capacity

Decision Variables:

- x_i : production in month i
- y_i : inventory at end of month i

Objective:

$$\min \sum_{i \in M} C_i x_i + H y_i \quad (\text{production} + \text{inventory cost})$$

Constraints:

$$\begin{aligned} y_1 &= x_1 - D_1 \\ y_i &= y_{i-1} + x_i - D_i \quad \forall i \in M, i \geq 2 \\ 0 &\leq x_i \leq K \quad \forall i \in M \\ 0 &\leq y_i \leq S \quad \forall i \in M \end{aligned}$$

Extensions:

- Expensive production or storage beyond thresholds (additional variables)
- Inventory shrinkage (e.g., $y_i = 0.98y_{i-1} + x_i - D_i$)
- Tiered holding cost or lead-time models

Python Implementation:

- Sets: `Months = [1, 2, 3, 4]`
- Parameters: `Demand[i]`, `ProductionCost[i]`, `HoldingCost`, `StorageCapacity`
- Variables: `x[i]`, `y[i]` (or indexed with tuples if more complex)

3.4 Live Session: Cash Flow Matching and Workforce Optimization

Cash Flow Matching Problem

The cash flow matching problem is a temporal variant of portfolio optimization. The goal is to invest in a set of bonds today to ensure liabilities over the next few years are met exactly, while minimizing total investment cost.

Setup:

- B : set of bonds
- T : set of years $\{1, 2, 3\}$
- p_i : price of bond i
- r_{it} : return from bond i in year t

- L_t : liability due in year t

Decision Variables:

- x_i : amount invested in bond i
- b_t : surplus balance carried into year t (auxiliary)

Objective:

$$\min \sum_{i \in B} p_i x_i$$

Constraints:

$$\begin{aligned} b_1 &= \sum_{i \in B} r_{i1} x_i - L_1 \\ b_t &= 1.01 \cdot b_{t-1} + \sum_{i \in B} r_{it} x_i - L_t \quad \forall t \geq 2 \\ b_t &\geq 0 \quad \forall t \in T \\ x_i &\geq 0 \quad \forall i \in B \end{aligned}$$

Notes:

- This model is structurally identical to an inventory model with holding cost (here, interest income).
- Surplus balances carry over with interest, enforcing liquidity.
- Gurobi implementation uses dictionaries for prices, returns, and liabilities.

Case Study: Workforce Optimization with Predictive Analytics

Based on *Chapados et al. (2014)*, this case integrates machine learning-based sales forecasting with prescriptive optimization for staff scheduling.

Key Points:

- Retail context: 15 stores in Canada; 20 months of 30-minute interval data
- Predictive layer: sales curves estimated from forecasted traffic, item sales, and pricing
- Prescriptive layer: shift creation and worker assignment via integer linear programming

Optimization Model Highlights:

- Variables determine shift patterns and assign staff
- Revenue contribution of a schedule is fixed (from predictive model)
- Constraints include:
 - Labor and union rules

- Shift granularity (15-min intervals)
- Mandatory breaks and operational requirements

Impact:

- Generated schedules yield 2–3% improvement in net profit
- Demonstrates real-world benefit of combining machine learning forecasts with LP

Literature Connection

Chapados et al. (2014) propose a two-stage approach:

1. **Sales Forecasting:** Nonlinear predictive models generate sales curves as a function of number of workers
2. **Optimization:** A linear program uses the forecasted revenue per staffing level to choose optimal schedules

Reference: Chapados, N., Joliveau, M., L'Ecuyer, P., Rousseau, L.-M. (2014). Retail store scheduling for profit. *European Journal of Operational Research*, 239(3), 609–624.

4 Network Flow Models

4.1 Theory & Terminology

Network flow models provide a powerful framework for representing and solving logistics, routing, and connectivity problems. In these models, the system is abstracted as a directed graph composed of nodes and arcs:

- **Nodes:** represent sources, sinks, or intermediate points (e.g., warehouses, terminals, intersections).
- **Arcs:** represent connections or routes along which flow (e.g., goods, data, people) can occur.

Each arc can have the following attributes:

- **Unit cost:** cost per unit of flow
- **Lower and upper bounds:** limits on allowable flow
- **Flow variable x_{ij} :** amount of flow from node i to node j

Each node has a net **supply or demand**:

- $b_i > 0$: supply node
- $b_i < 0$: demand node
- $b_i = 0$: transshipment or intermediate node

Core Modeling Principle: Flow Conservation

At each node, the total incoming flow minus the total outgoing flow must equal the node's supply or demand:

$$\sum_{j:(j,i) \in A} x_{ji} - \sum_{j:(i,j) \in A} x_{ij} = b_i$$

This single structural constraint underlies all network flow models, including transportation, shortest path, max flow, and multicommodity problems.

Key Theoretical Properties

- **Integrality:** If all data are integer (supply/demand, bounds, costs), then the LP solution is guaranteed to be integer-valued.
- **Solvability:** Network flow problems can be solved very efficiently via specialized algorithms (e.g., network simplex, Dijkstra, push-relabel).
- **Modeling Power:** Network models capture a wide variety of applications using just flow conservation, capacity, and cost.

Application Domains

- **Logistics:** DC placement, vehicle routing, pipeline flow
- **Telecommunications:** call routing, bandwidth allocation
- **Social Networks:** influence propagation, fraud detection
- **Transportation:** shortest paths, emergency services

Worked Example: Mini Network

- 5 nodes, 7 arcs
- Mixed supply/demand structure: $b_1 = -30$, $b_2 = -15$, $b_3 = 15$, $b_4 = 20$, $b_5 = 10$
- Objective: minimize total cost of flow satisfying all demands

Objective:

$$\min 8x_{12} + 15x_{13} + 13x_{23} + 11x_{34} + 9x_{42} + 24x_{53} + 7x_{54}$$

Constraints:

$$\begin{aligned} x_{12} - x_{13} &= -30 \\ x_{12} + x_{42} - x_{23} &= 15 \\ x_{13} + x_{23} + x_{53} - x_{34} &= 20 \\ x_{34} + x_{54} - x_{42} &= 10 \\ x_{53} - x_{54} &= -15 \\ x_{ij} &\geq 0 \quad \text{for all arcs} \end{aligned}$$

Python Implementation Tips

- Sets: Nodes, Arcs
- Parameters: `Supply[i]`, `UnitCost[i,j]`
- Variables: `x[i,j]` for all arcs
- See `example_network.ipynb` for a full implementation

4.2 Transshipment Problem

The **Transshipment Problem** extends the transportation model by including intermediate nodes where goods can be temporarily held before reaching their final destination. These nodes have zero net supply or demand but enable cost-efficient routing.

Setup and Example

Consider two plants (nodes 1, 2), two warehouses (nodes 3, 4), and two customers (nodes 5, 6). Products may be shipped directly or via transshipment.

Supply/Demand:

- Plant 1: supply of 160
- Plant 2: supply of 200
- Customer 5: demand of 180
- Customer 6: demand of 180
- Warehouses (3 and 4): transshipment nodes

Formulation:

$$\begin{aligned}
 \min \quad & \sum_{(i,j) \in A} c_{ij} x_{ij} \\
 \text{s.t.} \quad & \sum_{j:(i,j) \in A} x_{ij} - \sum_{j:(j,i) \in A} x_{ji} = b_i \quad \forall i \in N \\
 & x_{ij} \geq 0 \quad \forall (i,j) \in A
 \end{aligned}$$

Remarks:

- Compact formulation uses supply/demand balance b_i at each node.
- Directionality is preserved via arc definitions.
- Flow balance constraints naturally accommodate intermediate routing.

Python Implementation:

- Sets: Nodes, Arcs
- Parameters: Supply[i], UnitCost[i,j], LowerBound[i,j], UpperBound[i,j]
- Variables: x[i,j]

4.3 Shortest Path Problem

This problem identifies the least-cost path from a source node s to a destination node t . It is a special case of the transshipment problem with:

- $b_s = -1$ (supply)
- $b_t = +1$ (demand)
- $b_i = 0$ for all intermediate nodes

Example: Shortest Flight Path

Travel from Providence (PVD) to Honolulu (HNL) with the shortest distance. Arcs represent flight legs with known distances.

Variables:

- $x_{ij} \in \{0, 1\}$: 1 if flight leg (i, j) is taken

Objective:

$$\min \sum_{(i,j) \in A} d_{ij} x_{ij}$$

Flow Conservation Constraints:

$$\begin{aligned} \sum_j x_{ij} - \sum_j x_{ji} &= -1 && \text{if } i = \text{PVD} \\ &= 1 && \text{if } i = \text{HNL} \\ &= 0 && \text{otherwise} \end{aligned}$$

Application: Equipment Replacement

This models long-term investment in capital machinery as a shortest path through years. Arcs encode cost of operating a machine for 1–3 years or buying new.

Setup:

- Nodes: years 1 through 6
- Arcs: (i, j) represent use of a machine from year i to j , with costs depending on duration

Objective: Minimize total cost over 6 years using a shortest path.

Implementation Notes:

- Set of arcs derived from feasible operation durations
- Cost for each arc is fixed cost + operating cost
- LP or network model yields integer solution without binary enforcement

4.4 Maximum Flow Problem

The **Maximum Flow Problem** involves maximizing the amount of flow that can pass from a source node s to a sink node t through a network, subject to capacity constraints on each arc.

Model Components

- N : set of nodes
- A : set of directed arcs (i, j)
- c_{ij} : capacity of arc (i, j)
- x_{ij} : flow on arc (i, j) (decision variable)

Objective:

$$\max \sum_{j:(s,j) \in A} x_{sj} - \sum_{j:(j,s) \in A} x_{js} \quad (\text{net outflow from source})$$

Constraints:

$$\begin{aligned} \sum_{j:(i,j) \in A} x_{ij} - \sum_{j:(j,i) \in A} x_{ji} &= 0 \quad \forall i \in N \setminus \{s, t\} && (\text{flow conservation}) \\ x_{ij} &\leq c_{ij} \quad \forall (i, j) \in A && (\text{capacity}) \\ x_{ij} &\geq 0 \quad \forall (i, j) \in A \end{aligned}$$

Example: Telecom Routing

- Nodes: cities (e.g., DC to SF)
- Arcs: telecom segments with capacities (in thousands of calls)
- Goal: Maximize number of calls from DC to SF

Implementation Notes

- All nodes have net supply = 0
- Variables x_{ij} for each arc
- Parameters: `Capacity[i,j]` for all (i, j)

4.5 Multi-Commodity Flow Problem

The **Multi-Commodity Flow Problem** extends the flow model by considering multiple commodities (types of flow) that share the same network infrastructure but have different origins and destinations.

Model Components

- N : set of nodes
- A : set of arcs
- K : set of commodities (e.g., shipments)
- c_{ij} : unit shipping cost on arc (i, j)

- u_{ij} : capacity of arc (i, j)
- s_{ik} : net supply/demand at node i for commodity k
- x_{ijk} : flow on arc (i, j) for commodity k

Objective:

$$\min \sum_{(i,j) \in A} \sum_{k \in K} c_{ij} x_{ijk} \quad (\text{total shipping cost})$$

Constraints:

$$\begin{aligned} \sum_{j:(i,j) \in A} x_{ijk} - \sum_{j:(j,i) \in A} x_{jik} &= s_{ik} \quad \forall i \in N, \forall k \in K && (\text{commodity flow balance}) \\ \sum_{k \in K} x_{ijk} &\leq u_{ij} \quad \forall (i, j) \in A && (\text{shared capacity}) \\ x_{ijk} &\geq 0 \quad \forall (i, j) \in A, \forall k \in K \end{aligned}$$

Example: Package Delivery

- Commodities: 4 delivery demands (e.g., 200 units from node 1 to 7)
- All arcs have shared capacity of 300 units
- Unit costs vary across arcs

Implementation Notes

- Sets: Nodes, Arcs, Commodities
- Parameters: Supply[i,k], Cost[i,j], Capacity[i,j]
- Variables: $x[i,j,k]$ for each arc and commodity

Note: If the shared capacity constraint is removed, each commodity problem can be solved independently.

4.6 Live Session: LNG Tanker Scheduling

This case study illustrates the power of network flow modeling to optimize large-scale scheduling decisions in maritime logistics. The objective is to determine the **minimum number of LNG vessels** required to fulfill all deliveries on their specified dates.

Problem Context

- Each order is a full tanker shipment to a terminal.
- Delivery dates are fixed; early or late arrivals are not allowed.
- Ships return to Qatar to reload after every delivery.
- Vessels may wait between deliveries, and may begin at any delivery.

Network Representation

- Let O be the set of delivery orders.
- Let $V = O \cup \{s, t\}$ be the node set, with s (source) and t (sink).
- Arcs:
 - $(i, j) \in A$ if a vessel can deliver j after i
(i.e., $W_i + D_i + D_j \leq W_j$)
 - (s, i) for all $i \in O$: starting delivery
 - (i, t) for all $i \in O$: ending delivery
 - (t, s) : flow from sink to source (total vessels used)

Decision Variables

- $x_{ij} \geq 0$: 1 if delivery j is made directly after delivery i , with restocking in between

Objective

$$\min x_{ts} \quad (\text{minimize the number of vessels})$$

Constraints

$$\begin{aligned} \sum_{(i,j) \in A} x_{ij} - \sum_{(j,i) \in A} x_{ji} &= 0 & \forall j \in V & \quad (\text{flow conservation}) \\ \sum_{(i,j) \in A} x_{ij} &= 1 & \forall j \in O & \quad (\text{every order fulfilled}) \\ x_{ij} &\geq 0 & \forall (i,j) \in A & \end{aligned}$$

Example Result

- Manual terminal-by-terminal scheduling required 21 vessels.
- The LP model yields an optimal solution with only **17 vessels**.
- Total wait time: 100 days Schedule duration: 643 days

Secondary Objective: Minimize Wait Time

Once the number of vessels is fixed, we may minimize total system wait time:

- Introduce weight w_{ij} for each arc: the wait time if going from i to j
- Fix $x_{ts} = 17$ (from primary objective)
- New objective: $\min \sum_{(i,j) \in A} w_{ij} x_{ij}$

Implementation Notes

- Raw data: delivery dates, port distances, vessel rules
- Gurobi model includes arc generation logic and LP formulation
- Python visualizations: network graph and Gantt charts

Outcomes

- Wait time reduced to 42 days Avg. wait per vessel: 4.61%
- Demonstrates practical use of nonstandard network flow formulation

Reference Model

See: `tanker_scheduling_model.pdf` for the full linear program and arc conditions.

5 Integer Programming

Integer Programming (IP) extends Linear Programming (LP) by enforcing that some or all decision variables must take integer or binary values. This allows the model to capture discrete decisions, such as on/off selections, yes/no choices, or counts of indivisible resources.

5.1 Methodology

IP vs. LP

- LP variables can take any real value within bounds.
- IP variables are restricted to integers, often binary.

Types of IP Models

- **Pure Integer Programming (PIP)**: All decision variables are integer.
- **Mixed Integer Programming (MIP)**: Some variables are continuous, others are integer.
- **Binary Integer Programming (BIP)**: All integer variables are restricted to 0 or 1.

Formulation Example: Capital Budgeting

- Objective: Select a set of projects to maximize total profit within a budget.

Decision Variables:

$$x_i = \begin{cases} 1 & \text{if project } i \text{ is selected} \\ 0 & \text{otherwise} \end{cases}$$

Objective:

$$\max \sum_i p_i x_i \quad (\text{maximize total profit})$$

Constraint:

$$\sum_i c_i x_i \leq B \quad (\text{total cost must not exceed budget})$$

Solving IPs: Branch-and-Bound (B&B)

- Solve the LP relaxation of the IP (ignore integrality).
- If the solution is fractional, choose a fractional variable x_i and create two subproblems: $x_i \leq \lfloor x_i^* \rfloor$ and $x_i \geq \lceil x_i^* \rceil$.
- Repeat recursively. Use bounding: if the LP bound is worse than the current best integer solution, prune the branch.
- Search tree continues until all integer-feasible solutions are considered or pruned.

Why Not Just Round?

- Rounding LP solutions can violate constraints.
- Feasible rounded solutions are often far from optimal.
- IP structure is combinatorial—requires exact enumeration strategies.

5.2 Set Covering Problem

The set covering problem models scenarios where a set of entities (e.g., facilities, crews) must collectively cover all demand points (e.g., districts, tasks) with minimum cost.

Mathematical Formulation

- Let I be the set of locations (e.g., possible fire station sites).
- Let J be the set of coverage requirements (e.g., districts).
- Let $N_j \subseteq I$ be the subset of locations that can cover j .

Decision Variables:

$$x_i = \begin{cases} 1 & \text{if a facility is opened at location } i \\ 0 & \text{otherwise} \end{cases}$$

Objective:

$$\min \sum_{i \in I} c_i x_i \quad (\text{minimize total cost})$$

Constraints:

$$\sum_{i \in N_j} x_i \geq 1 \quad \forall j \in J \quad (\text{each requirement must be covered})$$

Applications and Insights

- Facility location planning (fire stations, clinics, service centers).
- Redundancy in critical systems (backup generators, network routing).
- Coverage is guaranteed but not necessarily minimized in overlap.
- Can be extended to multi-covering and capacitated versions.

Implementation Notes

- Sets: `Districts`, `Neighbors[j]`
- Parameters: `Cost[i]`
- Variables: `x[i]` binary
- See `set_covering.ipynb` for full implementation and visualization

5.3 Knapsack Problem and Logical Rules

The knapsack problem models situations in which we must choose a subset of items, each with a value and weight (or cost), subject to a capacity constraint, to maximize total value.

Mathematical Formulation (Project Selection)

Let P be the set of projects.

- $x_i = 1$ if project i is selected, 0 otherwise
- N_i : Net Present Value (NPV) of project i
- R_i : required programmers for i
- C_i : capital required for i
- M : total available programmers
- B : total capital budget

Objective:

$$\max \sum_{i \in P} N_i x_i$$

Constraints:

$$\begin{aligned} \sum_{i \in P} R_i x_i &\leq M \\ \sum_{i \in P} C_i x_i &\leq B \\ x_i &\in \{0, 1\} \quad \forall i \in P \end{aligned}$$

Logical Rules as Linear Constraints

- If projects 2 and 6 cannot both be taken: $x_2 + x_6 \leq 1$
- If project 3 requires project 4: $x_4 \geq x_3$
- If projects 2 and 4 are all-or-none: $x_2 = x_4$
- If project 5 requires 1 and 2: $x_5 \geq x_1 + x_2 - 1$
- If at least one of 1, 2, 5 must be taken: $x_1 + x_2 + x_5 \geq 1$
- To forbid $(x_4, x_5) = (1, 0)$: $x_4 - x_5 \leq 0$
- If taking one of $\{1, 2, 3\}$ requires at least two of them: multiple linear constraints (see slide logic)

Python Implementation Notes

- Sets: `Projects`, `Rules`
- Parameters: `NPV[i]`, `ReqProgrammers[i]`, `Capital[i]`
- Variables: `x[i]` binary
- Rules matrix for constraint representation (`Combination[r,i]`)
- See `knapsack.ipynb` and `knapsack_rules.ipynb` for implementation

5.4 Fixed Cost Problem

Fixed cost models capture situations where initiating a resource incurs a fixed setup cost in addition to per-unit production cost. These are common in manufacturing and operations settings.

Example: Circuit Board Production

We must produce 1800 units using a set M of machines.

- $x_i = 1$ if machine i is used, 0 otherwise
- y_i = number of boards produced on machine i
- F_i = fixed cost for using machine i
- c_i = variable cost per board on machine i
- K_i = capacity of machine i
- D = total demand (1800 boards)

Objective:

$$\min \sum_{i \in M} F_i x_i + c_i y_i$$

Constraints:

$$\begin{aligned} \sum_{i \in M} y_i &= D \\ y_i &\leq K_i x_i \quad \forall i \in M \\ x_i &\in \{0, 1\}, \quad y_i \geq 0 \quad \forall i \in M \end{aligned}$$

Implementation Notes

- Sets: `Machines`
- Parameters: `FixedCost[i]`, `VariableCost[i]`, `Capacity[i]`, `Demand`
- Variables: `x[i]`, `y[i]`
- See `fixed_cost.ipynb` for solution

5.5 Live Session: Combinatorial Auction – Landing Slot Allocation

This case study illustrates a real-world integer programming (IP) application: determining the winning bids in a multi-item auction involving landing slots at a regional airport.

Problem Context

- A regional airport has 6 landing slots to auction.
- Airlines submit bids for **bundles** of slots that align with their schedules.
- Each airline may win **at most one bid**.
- Goal: **maximize total revenue** from accepted bids.

Key Concepts

- **Combinatorial auction:** Bidders submit bids on combinations of items.
- **Packing problem:** Select disjoint bundles to maximize value.
- **Winner determination problem:** Choose a feasible subset of bids yielding max revenue.

Mathematical Formulation

Let B be the set of submitted bids.

- $x_b = 1$ if bid b is accepted, 0 otherwise
- $v_b =$ value of bid b
- $S_b =$ set of slots requested in bid b
- $A_b =$ airline associated with bid b

Objective:

$$\max \sum_{b \in B} v_b x_b \quad (\text{maximize auction revenue})$$

Constraints:

$$\begin{aligned} \sum_{b \in B: s \in S_b} x_b &\leq 1 && \forall \text{ slot } s \quad (\text{no slot can be sold twice}) \\ \sum_{b \in B: A_b = a} x_b &\leq 1 && \forall \text{ airline } a \quad (\text{one bid per airline}) \\ x_b &\in \{0, 1\} && \forall b \in B \end{aligned}$$

Extended Constraints (Logical Rules)

- If slots 4 and 5 are assigned to one airline, slots 2 and 3 cannot be assigned to another airline:
 $x_b + x_{b'} \leq 1$ where b includes $\{4,5\}$ and b' includes $\{2,3\}$.
- If Gamma Airlines is assigned slot 1, then OptAir cannot be assigned slot 3:
 $x_{\text{Gamma:1}} + x_{\text{OptAir:3}} \leq 1$

Python Implementation Notes

- See `auction.ipynb` and `auction_solved.ipynb` for full setup.
- Define a bid-slot matrix and airline-bid matrix.
- Use binary decision variables for each bid.
- Encode logical rules using additional linear constraints.

Industry Example: CombineNet

- CombineNet developed high-performance IP-based algorithms for combinatorial auctions.
- Focused on sourcing and transportation.
- Acquired by SciQuest (now JAGGAER) in 2013 for \$43M.
- Demonstrates commercial value of IP techniques in auction optimization.

6 Integer Programming Applications

This module covers practical applications of integer programming across production planning and strategic operations. These applications often involve fixed costs, binary decisions, and network-based flows.

6.1 Minimum Order Problem

The minimum order problem models situations where fixed costs are incurred when placing an order, and a minimum quantity must be purchased if the order is made. These arise in procurement, logistics, and batch production.

Theoretical Insight

The model captures both binary activation decisions (e.g., engage supplier or not) and continuous order quantities, creating a classical mixed-integer structure. Linking constraints ensure consistency between binary and continuous variables.

Decision Logic:

- If no order is placed from supplier s ($z_s = 0$), then $x_s = 0$.
- If supplier s is used ($z_s = 1$), then $x_s \geq L_s$ and $x_s \leq U_s$.

Formulation: Let S be the set of suppliers.

- x_s = amount of plasma ordered from supplier s
- $z_s = 1$ if supplier s is selected, 0 otherwise
- c_s = cost per unit from supplier s
- f_s = fixed cost for ordering from supplier s
- L_s, U_s = minimum and maximum allowable orders from s
- D = total demand

Objective:

$$\min \sum_{s \in S} (c_s x_s + f_s z_s)$$

Constraints:

$$\begin{aligned} \sum_{s \in S} x_s &\geq D \\ x_s &\geq L_s z_s \quad \forall s \in S \\ x_s &\leq U_s z_s \quad \forall s \in S \\ z_s &\in \{0, 1\}, \quad x_s \geq 0 \quad \forall s \in S \end{aligned}$$

Remarks:

- This model is nonlinear unless bounds are added.
- Model can handle supplier-based discounting with piecewise extension.

Implementation Notes:

- Sets: Suppliers
- Parameters: $\text{Cost}[s]$, $\text{FixedCost}[s]$, $\text{MinQty}[s]$, $\text{MaxQty}[s]$, Demand
- Variables: $x[s]$ continuous, $z[s]$ binary
- See `minimum_order.ipynb` for implementation and analysis

6.2 Facility Location Problem

Facility location models capture decisions about where to place facilities to serve customer demand most effectively, often under cost or capacity considerations.

Theoretical Insight

This model blends fixed cost activation decisions (binary) with continuous flow allocation (shipping). It forms the foundation for many logistics, warehouse, and emergency service applications.

Key Concepts:

- Trade-off between opening costs and transportation savings.
- Integrates spatial/geographic decision-making.
- Capacitated variants introduce binding upper limits on facilities.

Formulation: Let I be the set of candidate facility sites, and J the set of customers.

- $z_i = 1$ if facility i is opened
- x_{ij} = amount shipped from facility i to customer j
- c_{ij} = unit cost from i to j
- f_i = fixed cost for opening facility i
- d_j = demand from customer j
- u_i = capacity of facility i

Objective:

$$\min \sum_{i \in I} f_i z_i + \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij}$$

Constraints:

$$\sum_{i \in I} x_{ij} = d_j \quad \forall j \in J \quad (\text{customer demand met})$$

$$\sum_{j \in J} x_{ij} \leq u_i z_i \quad \forall i \in I \quad (\text{respect facility capacity})$$

$$x_{ij} \geq 0, \quad z_i \in \{0, 1\}$$

Variants:

- Budgeted facility opening
- Limited number of locations (`# facilities ≤ K`) *Coverage maximization instead of cost minimization*

Implementation Notes:

- Sets: `Facilities`, `Customers`
- Parameters: `Demand[j]`, `FixedCost[i]`, `Cost[i,j]`, `Capacity[i]`
- Variables: `x[i,j]` continuous, `z[i]` binary
- See `facility_location.A.ipynb` and `facility_location.B.ipynb` for coverage vs. cost optimization variants

6.3 Location and Distribution

The Location/Distribution problem extends the classical facility location model by integrating both fixed construction costs and variable distribution costs into a unified framework for network design.

Key Features

- Strategic placement of production facilities (e.g., plants).
- Assignment of customer demand to active facilities.
- Capacity limits and demand satisfaction.
- Minimization of total cost (fixed + variable).

Decision Variables:

- $x_i = 1$ if plant i is constructed, 0 otherwise
- y_{ij} = amount shipped from plant i to customer j

Parameters:

- c_i = fixed cost of building plant i
- K_i = capacity of plant i

- D_j = demand from customer j
- p_{ij} = production/transportation cost per unit from i to j

Objective:

$$\min \sum_i c_i x_i + \sum_i \sum_j p_{ij} y_{ij}$$

Constraints:

$$\begin{aligned} \sum_i y_{ij} &= D_j && \forall j \quad (\text{satisfy demand}) \\ \sum_j y_{ij} &\leq K_i x_i && \forall i \quad (\text{enforce capacity only if plant open}) \\ y_{ij} &\geq 0, \quad x_i \in \{0, 1\} \end{aligned}$$

Implementation Notes:

- Model scales to multi-echelon supply chains.
- Set definitions: `Plants`, `Customers`
- See `plant_location.ipynb` and `plant_location_solved.ipynb` for computation.

6.4 Vehicle Routing

The vehicle routing problem (VRP) aims to determine the optimal set of routes for a fleet of vehicles to service a set of customers, minimizing total travel cost or time.

Variants

- **TSP**: Single vehicle visiting all nodes once.
- **VRP**: Multiple vehicles, possibly subject to load, time, or other constraints.

Basic Formulation (TSP):

- $x_{ij} = 1$ if we travel from location i to j
- d_{ij} = distance from i to j

Objective:

$$\min \sum_i \sum_j d_{ij} x_{ij}$$

Constraints:

$$\sum_i x_{ij} = 1 \quad \forall j \quad (\text{enter each location once})$$

$$\sum_j x_{ij} = 1 \quad \forall i \quad (\text{leave each location once})$$

$$\text{Subtour elimination:} \quad x_{ij} + x_{ji} \leq 1 \quad \forall i \neq j$$

Advanced Model (Time-indexed network)

- $x_{ijt} = 1$ if travel from i to j occurs at time t
- Flow conservation constraints link x_{ijt} across time steps.
- Allows flexibility for additional constraints (e.g., time windows, pickups/deliveries).

Implementation Notes:

- Sets: `Locations`, `TimeSteps`
- Parameters: `Distance[i,j]`, network dimensions
- Variables: `x[i,j]` or `x[i,j,t]` binary
- See `routing.ipynb` and `routing_solved.ipynb` for application to grocery delivery

Remarks:

- Concorde solver excels at TSP.
- IP + heuristics often used in practice.
- Flexible modeling essential for real-world VRP variants.

6.5 Live Session: Advanced Vehicle Routing**6.5.1 Overview**

In this live session, we extend the classical vehicle routing problem (VRP) by incorporating:

- Time windows
- Precedence constraints
- Larger instance scalability

We begin with a five-customer grocery delivery example and then expand to larger variants involving delivery constraints and scheduling windows. The goal is to demonstrate how integer programming (IP) models can adapt to real-world delivery networks.

6.5.2 Network-Based IP Model

Sets:

- L = set of locations (including depot)
- $T = \{1, \dots, |L|\}$ = discrete time steps

Decision Variables:

- $x_{ijt} = 1$ if we travel from i to j at step t
- D_t = cumulative distance traveled by step t
- P_j = position at which location j is visited

Parameters:

- d_{ij} = distance from location i to j
- $[l_j, u_j]$ = time window for location j
- M = large constant (e.g., $M = \max_j u_j$)

Objective:

$$\min \sum_{i \in L} \sum_{j \in L} \sum_{t \in T} d_{ij} x_{ijt}$$

Constraints:

- **Start and End at Depot:**

$$\sum_{j \in L} x_{1jt=1} = 1, \quad \sum_{i \in L} x_{i1t=|L|} = 1$$

- **Visit Each Location Once:**

$$\sum_{i \in L} \sum_{t \in T} x_{ijt} = 1 \quad \forall j \in L$$

- **Flow Conservation:**

$$\sum_{i \in L} x_{ijt} = \sum_{i \in L} x_{jit+1} \quad \forall j \in L, \forall t < |L|$$

- **Time Window Satisfaction:**

$$D_t \geq l_j x_{ijt}, \quad D_t \leq u_j x_{ijt} + M(1 - x_{ijt}) \quad \forall i, j \in L, \forall t \in T$$

- **Allow Waiting:**

$$D_t \geq D_{t-1} + \sum_{i,j \in L} d_{ij} x_{ijt} \quad \forall t \in T \setminus \{1\}$$

- **Precedence Constraints:**

$$P_i \leq P_j - 1 \quad \forall (i, j) \in \text{precedence pairs}$$

$$P_j = \sum_{i \in L} \sum_{t \in T} t \cdot x_{ijt} \quad \forall j \in L \setminus \{1\}$$

Variable Types: $x_{ijt} \in \{0, 1\}$, D_t, P_j continuous

6.5.3 Scalability and Solver Settings

- Use `MIPFocus = 1` or `Cuts = 3` in Gurobi for faster feasible solutions
- Relax integrality for heuristics and rounding-based strategies
- The model scales up to:
 - ~ 50 locations (pure TSP)
 - $\sim 25\text{--}30$ locations (with time windows or precedences)

6.5.4 Real-World Applications

- **Supply Chain Logistics:** Delivery/pickup routes for medium-haul trucking (50+ stops)
- **Maintenance/Service Routing:** Snow plows, garbage collection (25–100 stops)
- **Last-Mile Delivery:** Amazon, USPS (100+ stops, real-time re-optimization)

6.5.5 Implementation Notes

- See `routing_solved.ipynb`, `routing_variant.ipynb`, and Excel data in `routing_larger.zip`
- The model uses a time-indexed flow graph, suitable for extensions such as:
 - Multi-vehicle routing
 - Load balancing
 - Service times and wait buffers
- Precedence is handled using position indexing and logical constraints.

7 Nonlinear Programming

Nonlinear programming (NLP) addresses optimization problems where the objective function and/or some constraints are nonlinear. These models capture a broader range of realistic relationships across domains such as pricing, production, finance, and logistics.

7.1 Theory & Terminology

7.1.1 Motivation and Scope

Nonlinear Programming (NLP) enables optimization in settings where relationships among variables are not linear. This includes nonlinear revenue functions, risk-return tradeoffs, production with economies of scale, and utility modeling.

7.1.2 Key Definitions and Concepts

- **Nonlinear Program (NLP):** Optimize $f(x)$ subject to $g_i(x) \leq b_i$, where f or any g_i is nonlinear.
- **Convex Function:** A function f is convex if:

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y), \quad \forall \lambda \in [0, 1]$$
- **Convex Set:** Any convex combination of feasible points remains feasible.
- **KKT Conditions:** Necessary (and under convexity, sufficient) conditions for optimality, generalizing Lagrangian multipliers.
- **Local vs. Global Optimum:** Convex problems ensure global optimality. Nonconvex may contain multiple local optima.

Karush-Kuhn-Tucker (KKT) Conditions Consider the problem:

$$\min f(x) \quad \text{s.t.} \quad g_i(x) \leq 0, \quad h_j(x) = 0$$

Define the Lagrangian:

$$L(x, \lambda, \mu) = f(x) + \sum_i \lambda_i g_i(x) + \sum_j \mu_j h_j(x)$$

The KKT conditions (first-order necessary for optimality) are:

$$\nabla_x L(x^*, \lambda^*, \mu^*) = 0 \quad (\text{Stationarity})$$

$$g_i(x^*) \leq 0, \quad h_j(x^*) = 0 \quad (\text{Primal feasibility})$$

$$\lambda_i^* \geq 0 \quad (\text{Dual feasibility})$$

$$\lambda_i^* g_i(x^*) = 0 \quad (\text{Complementary slackness})$$

Interpretation:

- Each λ_i can be viewed as a shadow price: how much the objective would improve if $g_i(x) \leq 0$ were relaxed.
- Complementary slackness ensures $\lambda_i > 0$ only if the constraint $g_i(x) = 0$ is active.

Applicability: These conditions apply when constraint qualifications (like LICQ or Slater's condition) hold. For convex programs, they are both necessary and sufficient for optimality.

7.1.3 First and Second Order Conditions in NLP

Optimization problems involving smooth nonlinear functions require formal conditions to characterize optimality. This section summarizes the foundational conditions that are required or sufficient to verify whether a point is a local or global optimum.

7.1.3.1 Problem Setup Consider the constrained nonlinear optimization problem:

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \leq 0 \quad \text{for } i \in I \\ & h_j(x) = 0 \quad \text{for } j \in J \end{aligned}$$

where f , g_i , and h_j are continuously differentiable.

7.1.4 First-Order Necessary Conditions (FONC)

If x^* is a local minimum and regularity conditions (constraint qualification) hold, then there exist multipliers $\lambda_i \geq 0$ and μ_j such that the Karush-Kuhn-Tucker (KKT) conditions are satisfied:

- **Stationarity:** $\nabla f(x^*) + \sum_{i \in I} \lambda_i \nabla g_i(x^*) + \sum_{j \in J} \mu_j \nabla h_j(x^*) = 0$
- **Primal feasibility:** $g_i(x^*) \leq 0$, $h_j(x^*) = 0$
- **Dual feasibility:** $\lambda_i \geq 0$
- **Complementary slackness:** $\lambda_i g_i(x^*) = 0$

7.1.5 Second-Order Necessary Conditions (SONC)

If x^* is a local minimum and the FONC conditions hold, then the following condition must also hold:

Let Z be the critical cone:

$$Z = \{d : \nabla h_j(x^*)^T d = 0, \nabla g_i(x^*)^T d \leq 0 \text{ for all } i \in A(x^*) \text{ with } \lambda_i > 0\}$$

Then for all $d \in Z$:

$$d^T \nabla_x^2 L(x^*, \lambda^*, \mu^*) d \geq 0$$

where $L(x, \lambda, \mu)$ is the Lagrangian function.

7.1.6 Second-Order Sufficient Conditions (SOSC)

Suppose the KKT conditions hold at x^* with multipliers λ^* and μ^* , and that for all $d \in Z$, $d \neq 0$, we have:

$$d^T \nabla_x^2 L(x^*, \lambda^*, \mu^*) d > 0$$

Then x^* is a strict local minimum.

7.1.7 Constraint Qualifications (CQs)

The KKT conditions and FONC/SONC hold under certain regularity assumptions. Common constraint qualifications include:

- **LICQ (Linear Independence CQ):** The gradients of active inequality constraints and equality constraints are linearly independent.
- **MFCQ (Mangasarian-Fromovitz):** Less strict than LICQ; allows for degeneracy in some gradients.
- **Slater's Condition:** For convex problems, if there exists a point that strictly satisfies all inequality constraints, KKT conditions are both necessary and sufficient.

7.1.7.1 Interpretation and Intuition

- **FONC:** Ensures there is no descent direction within the feasible set.
- **SONC:** Ensures flatness in feasible directions does not correspond to decreasing objective.
- **SOSC:** Ensures curvature is sufficient to verify a local minimum.

Geometric View: At an optimum, the level curve of the objective is tangent to the constraint boundary. This corresponds to a zero directional derivative along all feasible directions.

Example: For the unconstrained problem $\min f(x)$, the FONC condition becomes $\nabla f(x) = 0$, and the SOSC condition is $\nabla^2 f(x) > 0$ (positive definite Hessian).

7.1.8 Best Practices

- Always test FONC first (KKT), then verify second-order conditions.
- Use SOSC when convexity is unclear.
- Be cautious of boundary optima: check constraints' gradients.

7.1.9 Duality Theory in Nonlinear Programming

This section expands on theoretical principles and practical applications of NLP, with a focus on duality theory, economic interpretation, and representative models.

Consider the nonlinear optimization problem:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

Assume f , g_i , and h_j are continuously differentiable. The Lagrangian function is:

$$L(x, \lambda, \mu) = f(x) + \sum_{i=1}^m \lambda_i g_i(x) + \sum_{j=1}^p \mu_j h_j(x)$$

The **dual function** is defined as:

$$q(\lambda, \mu) = \inf_{x \in \mathbb{R}^n} L(x, \lambda, \mu)$$

The **dual problem** is:

$$\max_{\lambda \geq 0, \mu} q(\lambda, \mu)$$

*Key Properties of Duality

- **Weak Duality:** For any feasible x and any (λ, μ) such that $\lambda \geq 0$, we have:

$$q(\lambda, \mu) \leq f(x)$$

- **Strong Duality:** If the problem is convex and satisfies Slater's condition, then:

$$\max_{\lambda \geq 0, \mu} q(\lambda, \mu) = \min_{x \in \mathbb{R}^n} f(x)$$

7.1.9.1 Economic Interpretation of Lagrange Multipliers

- Each multiplier λ_i represents the **shadow price** of constraint $g_i(x) \leq 0$.
- It quantifies the rate of improvement in the optimal value of the objective function per unit increase in the right-hand side of the constraint.
- For equality constraints, μ_j plays a similar role.

Example: Suppose $g_1(x) = b - a^T x$ and $\lambda_1 = 5$. Then, relaxing the constraint b by one unit improves the objective by approximately 5 units.

7.1.9.2 Representative Applications of NLP *Portfolio Optimization (Convex QP)

$$\begin{aligned} \min_x \quad & x^T Q x \\ \text{s.t.} \quad & r^T x \geq \rho \\ & e^T x = 1, \quad x \geq 0 \end{aligned}$$

- Q is a positive semidefinite covariance matrix - r is expected return vector, ρ target return
- Convex QP with linear constraints

*Nonlinear Pricing (Nonconvex NLP)

$$\max_p \sum_{i=1}^n p_i D_i(p_i) \quad \text{subject to capacity and budget constraints}$$

- Demand $D_i(p_i)$ typically modeled as $D_i = a_i - b_i p_i$ or logit function - Results in concave revenue but possibly nonconvex constraints

*Regularized Regression (Convex QP)

$$\min_{\beta} \|Y - X\beta\|^2 + \lambda \|\beta\|_2^2$$

- Ridge regression - QP form with quadratic objective and closed-form solution if unconstrained

*Utility Maximization

$$\max_x \prod_{i=1}^n x_i^{\alpha_i} \quad \text{s.t.} \quad \sum_{i=1}^n p_i x_i \leq B$$

- Cobb-Douglas utility, nonlinearity in objective - Convex under $\alpha_i > 0$ and $\sum \alpha_i = 1$

7.1.9.3 Solvers for NLP

- **Gurobi, CPLEX:** Handle convex QP and some NLPs via QCP interfaces
- **IPOPT, SNOPT:** Interior point solvers for large-scale NLP
- **Pyomo, CVXPY, JuMP:** High-level modeling tools supporting nonlinear constraints
- **Heuristics (GA, SA):** Useful for nonconvex NLP where global solutions are needed

7.1.9.4 Summary and Best Practices

- Use duality to assess bounds and interpret constraints
- Check convexity and apply KKT + second-order conditions accordingly
- For convex NLP, leverage strong duality; otherwise use heuristics or global solvers

7.1.10 Convexity in Nonlinear Programming

Convexity is the central structure underpinning tractable nonlinear optimization. In this section, we provide a mathematically rigorous and practically motivated treatment of convexity as it appears in nonlinear programming (NLP).

7.1.10.1 Mathematical Foundations *Convex Sets A set $C \subseteq \mathbb{R}^n$ is *convex* if for any $x, y \in C$ and $\theta \in [0, 1]$:

$$\theta x + (1 - \theta)y \in C$$

This implies the line segment between any two points in C lies entirely within C .

*Convex Functions A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is *convex* on domain $\mathcal{D}$ if for all $x, y \in \mathcal{D}$ and $\theta \in [0, 1]$:

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$$

Strict convexity holds if the inequality is strict whenever $x \neq y$ and $\theta \in (0, 1)$.

First-Order Characterization: If f is differentiable, it is convex if and only if:

$$f(y) \geq f(x) + \nabla f(x)^T(y - x), \quad \forall x, y \in \mathcal{D}$$

This guarantees the epigraph of f lies above all its supporting hyperplanes.

Second-Order Characterization: If f is twice differentiable, then f is convex if and only if:

$$\nabla^2 f(x) \succeq 0 \quad \forall x \in \mathcal{D}$$

That is, the Hessian matrix is positive semidefinite.

7.1.10.2 Convex NLP Problem Form A standard convex NLP problem has the form:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

where:

- $f(x)$ and $g_i(x)$ are convex functions
- $h_j(x)$ are affine (i.e., $h_j(x) = a_j^T x - b_j$)

7.1.10.3 Key Properties and Implications

- **Local = Global Minimum:** Any local minimizer of a convex problem is also a global minimizer.
- **Strong Duality:** If Slater's condition holds, then the duality gap is zero.
- **KKT Sufficiency:** KKT conditions are both necessary and sufficient for optimality.

7.1.10.4 Examples of Convex NLPs

- **Quadratic Programming:** $\min x^T Q x + c^T x$ with $Q \succeq 0$
- **Logistic Regression:** $\min \sum_{i=1}^n \log(1 + e^{-y_i w^T x_i}) + \lambda \|w\|_2^2$
- **Portfolio Optimization:** Minimize variance with linear return and budget constraints

7.1.10.5 Nonconvex NLPs: Contrast and Challenge If $f(x)$ or any $g_i(x)$ are non-convex, then:

- Global optima are not guaranteed
- Multiple local optima may exist
- Duality gap may be nonzero
- Standard solvers may converge to suboptimal points

In such cases, global optimization, heuristics (e.g., simulated annealing, genetic algorithms), or careful initialization may be needed.

7.1.10.6 Testing Convexity in Practice

- **Hessian Check:** Symbolic or numerical evaluation of $\nabla^2 f(x)$
- **Eigenvalue Analysis:** All eigenvalues ≥ 0 implies convexity (PSD Hessian)
- **Convexity-Preserving Operations:** Recognize convex functions under affine compositions, max, pointwise supremum, etc.

7.1.10.7 Modern NLP Solver Implications

- Convexity enables reliable use of interior-point methods (e.g., IPOPT, MOSEK)
- Modeling frameworks (e.g., CVXPY, JuMP) exploit DCP (Disciplined Convex Programming) rules
- Certification of optimality is possible via dual residuals and complementary slackness

7.1.10.8 Conclusion and Best Practices

- Always verify convexity when applying KKT conditions or selecting solvers
- Prefer convex formulations for robustness and tractability
- Use symbolic tools (e.g., SymPy, Mathematica) or modeling packages to test convexity structure

Convexity remains the most foundational assumption for optimization theory, enabling both rigorous guarantees and computational feasibility across diverse applications in economics, engineering, data science, and operations research.

7.1.11 Selected Problem Types

- **Single Variable NLP:** Involves $f(x)$ with one decision variable. Uses derivative tests and domain evaluation.
- **Multivariate NLP:** $f(x_1, \dots, x_n)$ with nonlinear constraints. Solved via gradient-based or numerical methods.
- **Quadratic Programming (QP):** Special case where $f(x) = \frac{1}{2}x^T Qx + c^T x$, Q is positive semidefinite.

- **Mixed-Integer Nonlinear Programming (MINLP):** Combines integer constraints with nonlinear models.

7.1.12 Theory and Computation

Single Variable Optimization

- Find $f'(x) = 0$ to identify critical points.
- Use $f''(x)$ to determine local minima/maxima.
- Consider boundary points and domain constraints.

Multivariate Optimization

- First-order optimality: $\nabla f(x) = 0$ (stationarity)
- Second-order: Hessian matrix must be positive semidefinite.
- KKT conditions for constrained problems.

Convexity Check: f is convex if its Hessian $\nabla^2 f(x)$ is positive semidefinite for all x .

Quadratic Programming Example:

- Minimize portfolio variance subject to budget and return constraints.
- Model: $\min \frac{1}{2}x^T Qx$ s.t. $Ax \geq b, x \geq 0$
- Solved using CVXPY, Gurobi, or Excel GRG.

MINLP Example: Penetration Pricing

- Objective: $\min \sum x_i$ s.t. $\sum x_i a_i e^{b_i x_i} \geq R$
- Discretize $x_i \in \{1, 2, \dots, 25\}$ using binary variables y_{iv}
- Reformulate as MILP using precomputed r_{iv}

7.1.13 Solver Landscape

- **GRG Nonlinear:** Excel-based solver for smooth NLPs.
- **SQP (Sequential Quadratic Programming):** Iteratively solves QPs to approximate NLPs.
- **Evolutionary Algorithms:** Heuristic global search (non-deterministic).
- **MINLP Solvers:** COUENNE, BARON, BONMIN; combine NLP and integer programming methods.

7.1.14 Software and Tools

- Excel Solver: GRG and evolutionary algorithms.
- Python: `scipy.optimize`, `cvxpy`, `pyomo`, `gurobipy`

- Jupyter notebooks: `portfolio.ipynb`, `pricing.ipynb`, `regression.ipynb`

7.1.15 Modeling Tips

- Always check convexity before selecting solver.
- Use plots and derivatives to validate Solver solutions.
- For MINLP, ensure binary reformulation matches original nonlinear logic.
- Scale variables appropriately for numeric stability.

7.1.16 Common Pitfalls

- Solver convergence to local (not global) optima in nonconvex models.
- Overlooking constraints during manual differentiation.
- Using insufficient discretization granularity in MINLP linearization.
- Assuming QP models are always convex—check Q 's definiteness!

7.1.17 Gradient-Based Algorithmic Strategies

Gradient-based methods form the computational backbone for solving smooth nonlinear optimization problems. These methods leverage first-order or second-order derivative information to guide the search for a local minimum.

Steepest Descent (Gradient Descent) Algorithm:

$$x^{k+1} = x^k - \alpha_k \nabla f(x^k)$$

- α_k : step size or learning rate. - The direction $-\nabla f(x^k)$ is the steepest descent direction.

Notes: - Works best for convex, smooth functions. - Sensitive to choice of step size.

Newton's Method Algorithm:

$$x^{k+1} = x^k - [\nabla^2 f(x^k)]^{-1} \nabla f(x^k)$$

- Uses second-order curvature via the Hessian matrix. - Offers quadratic convergence near the optimum.

Notes: - More expensive per iteration than gradient descent. - May not converge if the Hessian is not positive definite.

Projected Gradient Descent (PGD) Algorithm:

$$x^{k+1} = \Pi_{\Omega}(x^k - \alpha_k \nabla f(x^k))$$

- Π_{Ω} : projection operator onto the feasible set Ω .

Notes: - Used for constrained problems. - Ensures feasibility of intermediate iterates.

Feasible Direction Method Idea: - At each iteration, search direction d^k is such that the constraints remain satisfied:

$$\nabla f(x^k)^T d^k < 0, \quad d^k \in T(x^k)$$

- $T(x^k)$: tangent cone to feasible set at x^k

Notes: - Feasibility maintained without explicit projection. - Generalizes well to nonlinear inequality constraints.

Line Search and Backtracking - Used to determine α_k . - Ensures sufficient decrease (Armijo rule):

$$f(x^k - \alpha_k \nabla f(x^k)) \leq f(x^k) - c\alpha_k \|\nabla f(x^k)\|^2$$

- Common in both unconstrained and constrained optimization.

6. Summary of Methods

Method	Uses Hessian?	Handles Constraints?
Steepest Descent	No	No
Newton's Method	Yes	No
Projected Gradient	No	Yes (via projection)
Feasible Direction	No	Yes (implicit)

Remark: Gradient methods serve as the foundation for more advanced techniques such as Sequential Quadratic Programming (SQP), Interior Point Methods, and Trust-Region Approaches.

7.2 Single Variable NLP

Motivation and Overview

Many practical problems involve nonlinear relationships that can't be modeled using linear programming. Examples include pricing models, demand curves, and exponential growth.

Business Applications: Pricing optimization, revenue modeling, marketing elasticity.

Theory: Calculus-Based Optimization

To find local optima:

- Solve $f'(x) = 0$ to find critical points.
- Use second derivative $f''(x)$ to classify the critical point:
 - $f''(x^*) > 0$ implies local minimum
 - $f''(x^*) < 0$ implies local maximum
- Also check domain boundaries and nondifferentiable points.

Convexity: A convex function has a global minimum if $f''(x) \geq 0$ for all x in domain. Concave functions behave analogously for maximization.

Example: Revenue Maximization Given demand $f(x) = ae^{-0.2x}$, revenue is $g(x) = xf(x)$. Solve:

$$g'(x) = ae^{-0.2x}(1 - 0.2x) = 0 \Rightarrow x^* = 5$$

$$g''(x) = ae^{-0.2x}(0.04x - 0.4) \Rightarrow g''(5) < 0 \Rightarrow \text{maximum}$$

Domain Considerations: Always analyze the full feasible domain to confirm global optimality.

Numerical Methods: Excel Solver

Solver Features:

- GRG Nonlinear for smooth nonlinear models
- Evolutionary for nonsmooth problems
- Requires cell-based formulation: decision variables, objective, constraints

Pros: Easy to set up and interpret. Good for small models and quick prototyping.

Cons: May return local optima only. Not suitable for large-scale or symbolic models.

Example Workbook: See `nlp.xlsx` for implementation.

7.3 Multivariate NLP

Motivation and Applications

Multivariate NLP arises in:

- Portfolio optimization (risk-return tradeoffs)
- Manufacturing (nonlinear cost/quality tradeoffs)
- Pricing and bundling strategies

Mathematical Formulation

$$\begin{aligned} \min \quad & f(x_1, x_2, \dots, x_n) \\ \text{s.t.} \quad & g_i(x_1, \dots, x_n) \leq b_i \quad \forall i = 1, \dots, m \end{aligned}$$

Classification of Models:

- Convex vs. Nonconvex
- Smooth (differentiable) vs. Nonsmooth
- With or without integer variables (MINLP)

Solution Techniques

- Gradient-based solvers (e.g., GRG)
- Global optimization heuristics
- Piecewise linearization (for approximation)
- Sequential Quadratic Programming (SQP)

Example: Portfolio Optimization

- Variables: investment levels in assets $x_1, x_2, \dots$
- Objective: Minimize risk (quadratic form $x^T Q x$)
- Constraints: Total budget, minimum return, asset bounds

Karush-Kuhn-Tucker (KKT) Conditions:

- Necessary optimality conditions for constrained nonlinear programs
- Generalization of Lagrangian multipliers for inequality-constrained problems

Tools: Excel Solver, Python (SciPy.optimize, CVXPY), Gurobi (for convex QP)

Example Workbook: See `nlp_solved.xlsx` for implementation and model diagnostics.

7.4 Quadratic Programming

Overview

Quadratic programming (QP) problems involve a quadratic objective function and linear constraints. QP models appear frequently in finance, regression, and engineering design.

Standard Form:

$$\begin{aligned} \min_x \quad & \frac{1}{2} x^T Q x + c^T x \\ \text{s.t.} \quad & A x \geq b \end{aligned}$$

Where Q is symmetric and positive semidefinite for convex problems.

Applications:

- Least-squares regression
- Portfolio optimization (Markowitz model)
- SVM classification (machine learning)

Theory: Convexity and Optimality

- If Q is positive semidefinite, the problem is convex and any local minimum is global.
- If Q is indefinite, nonconvex QP can have multiple local minima—solvers may return suboptimal solutions.
- KKT conditions still apply: stationarity, primal feasibility, dual feasibility, complementary slackness.

Regression Example

Fit a model $y = ax^2 + bx + c$ to minimize sum of squared errors:

- Decision variables: a, b, c
- Objective: $\min \sum_{i=1}^n (ax_i^2 + bx_i + c - y_i)^2$
- Convex in nature; can be solved by setting gradient $\nabla f = 0$ or using optimization solvers.

Implementation: See `regression.ipynb`. The fitted model: $y = 1.1459x^2 - 2.8002x + 5.0515$

Portfolio Optimization (Markowitz)

- Variables: x_i = fraction of capital invested in stock i
- Objective: $\min \sum_{i,j} \sigma_{ij} x_i x_j$ (portfolio variance)
- Constraints: $\sum x_i = 1, \sum r_i x_i \geq R, x_i \geq 0$
- Matrix form: $\min \frac{1}{2} x^T Q x$ s.t. $Ax \geq b$

Tools: Python (CVXPY, Gurobi), Excel Solver. See `portfolio.ipynb` and `portfolio.xlsx`.

7.5 General Integer NLP (MINLP)

Model Overview

Mixed-Integer Nonlinear Programming (MINLP) problems allow nonlinearities in objective and/or constraints and also include discrete variables.

General Form:

$$\begin{aligned} \min_{x,y} \quad & f(x, y) \\ \text{s.t.} \quad & g_i(x, y) \leq b_i, \quad x \in \mathbb{R}^n, y \in \mathbb{Z}^m \end{aligned}$$

Theory and Challenges

- MINLPs are NP-hard and may contain both continuous nonconvexity and combinatorial complexity.

- Strong duality generally does not hold unless convex and continuous.
- Global optima require exact methods (e.g., branch-and-bound, spatial B&B).

Solution Methods

- Gradient-based solvers: fast but local (e.g., IPOPT)
- Genetic algorithms: heuristic search with no optimality guarantee
- Branch-and-bound with nonlinear relaxations: global but slower

Solvers: IPOPT, Couenne, Baron, MIDACO, Pyomo, GAMS

Linearization Strategy

- Convert integer nonlinearities using binary expansion
- Pre-compute nonlinear terms (e.g., $r_{iv} = va_ie^{b_iv}$)
- Reformulate constraints and objective linearly using $y_{iv} \in \{0, 1\}$

Example: Penetration Pricing

- Objective: Minimize total prices $\sum_i x_i$ subject to $\sum_i x_i a_i e^{b_i x_i} \geq R$
- $x_i \in \{1, 2, \dots, 25\}$ discretized using $y_{iv} \in \{0, 1\}$
- Final LP formulation:

$$\begin{aligned} & \min \sum_{i \in P} \sum_{v \in V} v y_{iv} \\ \text{s.t. } & \sum_{v \in V} y_{iv} = 1 \quad \forall i, \quad \sum_{i \in P} \sum_{v \in V} r_{iv} y_{iv} \geq R \end{aligned}$$

Implementation: See `pricing.ipynb` and `pricing.xlsx`.

Remarks

- Linearization allows standard MIP solvers to handle complex MINLPs
- Significant scalability improvement vs. gradient or genetic methods
- Trade-off: linearization introduces size and approximation error

7.6 Barrier and Interior-Point Methods for Nonlinear Programming

Barrier and interior-point methods are indispensable tools for solving large-scale constrained nonlinear programming (NLP) problems. Barrier and interior-point methods combine deep convex theory with powerful numerical properties, making them indispensable in large-scale,

structured NLP. These methods are not only theoretically elegant, drawing from duality and convex analysis, but also practical in that they serve as the core of high-performance solvers such as IPOPT and KNITRO.

7.6.1 Theoretical Properties

- $\phi(x; \mu)$ is strictly convex if f and g_i are convex.
- The solution $x^*(\mu)$ remains in the interior of the feasible set.
- As $\mu \rightarrow 0$, $x^*(\mu)$ converges to the optimal solution of the original problem.

7.6.2 Barrier Method Formulation

Consider the standard NLP:

$$\min_x f(x) \quad \text{s.t.} \quad g_i(x) \leq 0, \quad h_j(x) = 0$$

Assume that f , g_i , and h_j are twice continuously differentiable.

We define the barrier-augmented objective as:

$$\phi(x; \mu) = f(x) - \mu \sum_{i=1}^m \log(-g_i(x))$$

subject to $h_j(x) = 0$ and $g_i(x) < 0$.

This formulation prevents infeasibility by penalizing constraint violations with an infinite barrier as $g_i(x) \rightarrow 0^-$.

We can also consider the inequality-constrained NLP:

$$\min_x f(x) \quad \text{s.t.} \quad g_i(x) \leq 0, \quad i = 1, \dots, m$$

Let each g_i be convex and f be convex and twice continuously differentiable. Define the **log-barrier function**:

$$\phi(x; \mu) = f(x) - \mu \sum_{i=1}^m \log(-g_i(x)), \quad \text{with } g_i(x) < 0$$

The central problem is now:

$$\min_x \phi(x; \mu)$$

As $\mu \rightarrow 0$, solutions to this problem trace a **central path** toward the solution of the original problem.

7.6.2.1 Barrier Properties

- $\phi(x; \mu)$ is convex if f and g_i are convex
- The iterates remain in the interior of the feasible region
- The exact constraint is approached asymptotically as $\mu \rightarrow 0$

7.6.3 Central Path and Barrier KKT Conditions

Let $x^*(\mu)$ solve the barrier problem for fixed μ . The trajectory $\{x^*(\mu)\}_{\mu>0}$ is the *central path*.

The KKT conditions for the barrier problem are:

$$\nabla f(x) + \sum_i \frac{\mu}{-g_i(x)} \nabla g_i(x) + \sum_j \lambda_j \nabla h_j(x) = 0 \quad \text{with } h_j(x) = 0$$

These approximate the true KKT conditions as $\mu \rightarrow 0$:

$$\nabla f(x) + \sum_i \lambda_i \nabla g_i(x) + \sum_j \lambda_j \nabla h_j(x) = 0, \quad \lambda_i \geq 0, \quad \lambda_i g_i(x) = 0$$

Theorem (Convergence of Central Path): If the feasible region has nonempty interior (Slater's condition holds), then the central path $x^*(\mu)$ exists for all $\mu > 0$ and converges to a solution x^* satisfying the KKT conditions.

7.6.4 Barrier KKT Conditions

For fixed $\mu > 0$, the KKT conditions for the barrier problem become:

$$\nabla f(x) + \sum_{i=1}^m \frac{\mu}{-g_i(x)} \nabla g_i(x) = 0$$

These approximate the true KKT conditions:

$$\nabla f(x) + \sum_i \lambda_i \nabla g_i(x) = 0, \quad \lambda_i \geq 0, \quad \lambda_i g_i(x) = 0$$

7.6.5 Primal-Dual Interior-Point Formulation

This formulation directly enforces complementarity through a perturbed system:

$$\begin{aligned} \nabla f(x) + \sum_i \lambda_i \nabla g_i(x) + \sum_j \mu_j \nabla h_j(x) &= 0 \\ \lambda_i g_i(x) &= -\mu, \quad \lambda_i > 0, \quad g_i(x) < 0 \\ h_j(x) &= 0 \end{aligned}$$

This system is solved using Newton-type updates on the residuals.

7.6.5.1 Residual Vector Definition Let $r_d(x, \lambda, \mu) = \nabla f(x) + J_g(x)^T \lambda + J_h(x)^T \mu$ be the dual residual, $r_p(x) = h(x)$ the primal residual, and $r_c(x, \lambda) = \text{diag}(\lambda)g(x) + \mu \mathbf{1}$ the complementarity residual.

7.6.6 Primal-Dual Newton Step

We compute Newton steps $\Delta x, \Delta \lambda, \Delta \mu$ from the linearized residual equations:

$$\nabla R(x, \lambda, \mu) \begin{bmatrix} \Delta x \\ \Delta \lambda \\ \Delta \mu \end{bmatrix} = - \begin{bmatrix} r_d \\ r_p \\ r_c \end{bmatrix}$$

This requires solving a symmetric indefinite system, often reduced via Schur complement.

7.6.7 Algorithmic Steps

1. Initialize $x^{(0)}$ strictly feasible, $\lambda^{(0)} > 0$, choose $\mu^{(0)}$
2. Repeat:
 - (a) Compute search direction $\Delta x, \Delta \lambda, \Delta \mu$
 - (b) Perform line search ensuring $x + \alpha \Delta x$ remains strictly feasible
 - (c) Update: $x \leftarrow x + \alpha \Delta x$, etc.
 - (d) Decrease $\mu \leftarrow \sigma \mu$ (e.g., $\sigma = 0.1$)
3. Terminate when residuals and duality gap are below tolerance

7.6.8 Convergence Properties

- Global convergence under mild assumptions (e.g., Slater's condition)
- Local quadratic convergence when using exact Hessians
- Fast convergence to high-precision KKT points

7.6.9 Comparison with Active Set Methods

- Interior-point avoids combinatorial growth in constraint activation
- Handles ill-conditioning more gracefully via centrality
- Active-set better when solution lies on boundary with few active constraints

7.6.10 Applications and Solver Use

- Used in IPOPT (filter-based globalization), MOSEK, and KNITRO
- Common in conic programming (e.g., SOCP, SDP), robust QPs, trajectory optimization, and portfolio optimization

7.6.11 Connection to Convex Analysis

- Assumes Slater's condition to ensure bounded duality gap
- Central path defines a trajectory of strictly feasible dual solutions
- Lagrange multipliers emerge naturally via perturbed complementarity

7.7 Sensitivity Analysis in Nonlinear Programming

Sensitivity analysis investigates how the optimal solution and objective value change in response to small perturbations in the problem data. In the context of nonlinear programming (NLP), sensitivity analysis extends concepts from linear programming but incorporates richer geometry and curvature. Sensitivity analysis in NLP provides deep insight into robustness and economic interpretation, guiding model design and policy under uncertainty.

7.7.1 Setup and Notation

Consider a parametric NLP:

$$\begin{aligned} \min_x \quad & f(x, \theta) \\ \text{s.t.} \quad & g_i(x, \theta) \leq 0, \quad i = 1, \dots, m \\ & h_j(x, \theta) = 0, \quad j = 1, \dots, p \end{aligned}$$

where $\theta \in \mathbb{R}^k$ is a vector of parameters.

Let $x^*(\theta)$ denote the optimal solution and $\lambda^*(\theta), \mu^*(\theta)$ the corresponding Lagrange multipliers.

7.7.2 Total Derivative of the Optimal Value

The optimal value function is defined as:

$$\varphi(\theta) = f(x^*(\theta), \theta)$$

Assuming differentiability and regularity conditions (e.g., LICQ, SOSC), we can compute its derivative using the chain rule:

$$\nabla_{\theta} \varphi(\theta) = \frac{\partial f}{\partial x} \frac{dx^*}{d\theta} + \frac{\partial f}{\partial \theta}$$

To eliminate $\frac{dx^*}{d\theta}$, we differentiate the KKT conditions implicitly.

7.7.3 Differentiation of KKT Conditions

Let $L(x, \lambda, \mu, \theta) = f(x, \theta) + \sum \lambda_i g_i(x, \theta) + \sum \mu_j h_j(x, \theta)$.

First-order optimality yields:

$$\nabla_x L(x^*, \lambda^*, \mu^*, \theta) = 0$$

Differentiate with respect to θ :

$$\frac{\partial^2 L}{\partial x^2} \cdot \frac{dx^*}{d\theta} + \frac{\partial^2 L}{\partial x \partial \theta} + \sum_i \frac{d\lambda_i^*}{d\theta} \nabla g_i(x^*, \theta) + \sum_j \frac{d\mu_j^*}{d\theta} \nabla h_j(x^*, \theta) = 0$$

This linear system can be solved for $\frac{dx^*}{d\theta}$ and used in $\nabla_{\theta} \varphi(\theta)$.

7.7.4 Shadow Prices and Reduced Costs in NLP

- λ_i^* interprets the sensitivity of the optimal value to tightening or relaxing inequality $g_i(x, \theta)$
- μ_j^* captures sensitivity to equality constraint perturbation
- Reduced cost generalizes to the directional derivative of the objective under a perturbation of an inactive constraint

7.7.5 Sensitivity of the Active Set

A small change in θ may change the set of active constraints. This is a non-smooth transition, requiring careful tracking using:

- Active set stability conditions
- Semi-smooth Newton or continuation methods

7.7.6 Applications and Use Cases

- Pricing constraints in power systems and network flow
- Scenario analysis in finance and logistics
- Stability analysis in multi-period and bilevel NLPs

7.7.7 Best Practices

- Ensure LICQ and SOSC hold before performing differential sensitivity
- Use symbolic differentiation or automatic differentiation for Jacobians
- Combine with solver output (e.g., duals from IPOPT) for local analysis

7.8 Advanced Newton and Quasi-Newton Methods in NLP

Newton-type methods form the backbone of many state-of-the-art nonlinear optimization algorithms. Their performance hinges on curvature, globalization, and numerical stability. Newton-type methods unify second-order information with robust step strategies. This section presents a mathematically rigorous and practically motivated treatment of Newton, Quasi-Newton, and Trust-Region methods. Leading solvers blend Newton theory with line search, trust region, and quasi-Newton heuristics to solve problems ranging from classical NLP to massive-scale ML.

7.8.1 Unconstrained Newton's Method

Consider the unconstrained optimization problem:

$$\min_{x \in \mathbb{R}^n} f(x)$$

Assuming f is twice continuously differentiable and convex, the Newton direction d_k at iteration k is:

$$d_k = -[\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$$

The update becomes:

$$x_{k+1} = x_k + \alpha_k d_k$$

where $\alpha_k \in (0, 1]$ is chosen via a line search procedure to ensure global convergence.

Descent Direction: d_k is a descent direction if $\nabla^2 f(x_k) \succ 0$. Otherwise, regularization or a trust-region strategy may be required.

7.8.2 Local Convergence Theory

- **Quadratic Convergence:** If f is strongly convex near x^* and $\nabla^2 f(x^*)$ is positive definite, then Newton's method converges quadratically in a neighborhood of x^* .
- **Damped Newton:** When $\alpha_k < 1$, convergence is linear or superlinear depending on how α_k is chosen (e.g., via Wolfe conditions).
- **Ill-conditioning:** When $\nabla^2 f(x_k)$ is nearly singular, Newton steps become unreliable unless regularized (e.g., $\nabla^2 f + \lambda I$).

7.8.3 Equality-Constrained Newton-KKT Method

For problems of the form:

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & h(x) = 0 \end{aligned}$$

The Newton step solves the KKT system:

$$\begin{bmatrix} \nabla_{xx}^2 L(x_k, \mu_k) & \nabla h(x_k)^T \\ \nabla h(x_k) & 0 \end{bmatrix} \begin{bmatrix} d_x \\ d_\mu \end{bmatrix} = - \begin{bmatrix} \nabla f(x_k) + \nabla h(x_k)^T \mu_k \\ h(x_k) \end{bmatrix}$$

This system is symmetric but indefinite. Solving it efficiently is a key challenge addressed by active-set, null-space, and interior-point approaches.

7.8.4 Quasi-Newton Methods: BFGS and L-BFGS

Motivation: Computing the full Hessian and its inverse is often expensive or unstable. Quasi-Newton methods iteratively approximate the inverse Hessian.

BFGS Update:

$$B_{k+1} = B_k - \frac{B_k s_k s_k^T B_k}{s_k^T B_k s_k} + \frac{y_k y_k^T}{y_k^T s_k}$$

where $s_k = x_{k+1} - x_k$, $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$.

Properties:

- Maintains positive-definiteness if $y_k^T s_k > 0$
- Superlinear convergence under standard conditions

L-BFGS: Stores only recent m pairs $\{s_k, y_k\}$ and applies a two-loop recursion. Used in large-scale ML optimization (e.g., logistic regression).

7.8.5 Trust-Region Newton Methods (TRON)

Rather than relying on line search, trust-region methods solve:

$$\min_d \quad m_k(d) = f(x_k) + \nabla f(x_k)^T d + \frac{1}{2} d^T B_k d \quad \text{s.t.} \quad \|d\| \leq \Delta_k$$

Highlights:

- More robust to poor curvature conditions
- Adaptive region resizing leads to better step selection
- Solvers: `trust-ncg`, `trust-krylov`, and IPOPT's primal-dual variant

7.8.6 Globalization Strategies

Line Search Methods:

- Armijo rule (sufficient descent)
- Wolfe and strong Wolfe conditions

Trust-Region Methods:

- Compute candidate step
- Evaluate actual vs. predicted reduction
- Accept/reject step and update radius

7.8.7 Implementation in Solvers

- **IPOPT:** Filter line search with BFGS updates or exact Hessians
- **KNITRO:** Newton, interior-point, active-set hybrid strategies
- **SciPy/PyTorch:** L-BFGS and trust-region options for ML

7.8.8 Best Practices

- Always verify descent direction (use $\nabla f(x)^T d < 0$)
- If Hessian is indefinite, try modified Cholesky or Levenberg-Marquardt
- Use Newton with exact Hessian only for small/medium-scale problems
- Use L-BFGS for large-scale sparse problems with noisy gradients
- Use trust-region strategies when conditioning or curvature is unreliable

7.9 Penalty and Augmented Lagrangian Methods for Nonlinear Programming

Penalty and augmented Lagrangian methods offer powerful alternatives to barrier methods for solving constrained nonlinear optimization problems. These methods bridge theory with numerics for real-world engineering and data applications by transforming constrained problems into a sequence of easier subproblems by incorporating constraint violations directly into the objective function. Penalty and augmented Lagrangian methods are deeply grounded in convex and variational analysis, offering practical advantages in large-scale and distributed NLPs.

7.9.1 Quadratic Penalty Method

Consider the equality-constrained problem:

$$\min_x f(x) \quad \text{s.t. } h(x) = 0$$

We construct the quadratic penalty function:

$$\phi(x; \rho) = f(x) + \frac{\rho}{2} \|h(x)\|_2^2 \quad \text{for } \rho > 0$$

Algorithm:

1. Initialize $x^{(0)}$ and $\rho_0 > 0$
2. For $k = 0, 1, \dots$ until convergence:
 - Solve $x^{k+1} = \arg \min \phi(x; \rho_k)$
 - Increase penalty: $\rho_{k+1} = \beta \rho_k$ with $\beta > 1$

Key Theoretical Properties:

- Convergence to feasible point as $\rho \rightarrow \infty$ under regularity assumptions
- Ill-conditioning increases with ρ , degrading numerical stability

7.9.2 Exact Penalty Methods (L1 Penalty)

An exact penalty formulation uses:

$$\phi(x; \rho) = f(x) + \rho \|h(x)\|_1$$

This formulation achieves constraint satisfaction at finite ρ under constraint qualification (e.g., LICQ).

Advantages:

- Does not require $\rho \rightarrow \infty$
- Can be minimized using non-smooth optimization techniques (e.g., subgradient or proximal methods)

7.9.3 Augmented Lagrangian Method (ALM)

Combines dualization with quadratic penalty:

$$\mathcal{L}_\rho(x, \lambda) = f(x) + \lambda^T h(x) + \frac{\rho}{2} \|h(x)\|_2^2$$

This results in the **Method of Multipliers**, iterating on x and Lagrange multipliers λ :

1. Initialize x^0 , λ^0 , and ρ
2. For $k = 0, 1, \dots$:
 - $x^{k+1} = \arg \min_x \mathcal{L}_\rho(x, \lambda^k)$
 - $\lambda^{k+1} = \lambda^k + \rho h(x^{k+1})$

Theoretical Guarantees:

- Saddle-point convergence under constraint qualifications (e.g., LICQ + second-order sufficiency)
- Local fast convergence for moderate ρ
- Widely used for large-scale decomposable optimization

7.9.4 Handling Inequality Constraints

Transform inequality constraints $g(x) \leq 0$ to equality using slack variables:

$$g(x) + s = 0, \quad s \geq 0$$

Apply ALM in variables (x, s) with bound constraints enforced via projection or interior penalties.

7.9.5 Connections to Dual Decomposition

ALM enables dual updates while stabilizing primal convergence:

- Enhances separability in distributed optimization
- Common in ADMM-style methods used in machine learning and signal processing

7.9.6 Implementation Challenges and Solutions

- Subproblem solvers must handle nonsmooth objectives (for exact penalties)
- Augmented systems require accurate gradients and Hessians
- Penalty update rules (e.g., Powell's strategy) are critical for efficiency

7.9.7 Comparative View with Barrier Methods

Feature	Penalty/ALM vs. Barrier
Feasibility Enforcement	Penalty: gradually enforced, Barrier: always interior
Numerical Stability	ALM: better conditioned, Barrier: sensitive near boundaries
Dual Variable Estimation	ALM: explicit via iteration, Barrier: implicit via μ path
Constraint Types	ALM: flexible for equality + inequality, Barrier: suited for strict inequality

7.9.8 Modern Software and Applications

- ALGECAN, LANCELOT, SNOPT: Classical NLP solvers
- `scipy.optimize`, `pyomo`, `cvxpy` with penalty interfaces
- Widely used in engineering design, robust control, and finance

7.10 Slater's Condition and Constraint Qualification

Slater's condition provides a sufficient condition for strong duality to hold in convex optimization. If the problem:

$$\begin{aligned}
 \min \quad & f(x) \\
 \text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m \\
 & h_j(x) = 0, \quad j = 1, \dots, p
 \end{aligned}$$

has a convex objective and inequality constraints, and there exists an *interior point* x^* such that:

$$g_i(x^*) < 0 \quad \text{for all } i = 1, \dots, m, \quad h_j(x^*) = 0 \quad \forall j,$$

then strong duality holds and the KKT conditions are necessary and sufficient for optimality.

7.10.1 KKT Conditions and Dual Feasibility

Given the Lagrangian $L(x, \lambda, \mu)$, the KKT conditions are:

1. **Stationarity:** $\nabla f(x^*) + \sum_{i=1}^m \lambda_i \nabla g_i(x^*) + \sum_{j=1}^p \mu_j \nabla h_j(x^*) = 0$
2. **Primal Feasibility:** $g_i(x^*) \leq 0, h_j(x^*) = 0$
3. **Dual Feasibility:** $\lambda_i \geq 0$
4. **Complementary Slackness:** $\lambda_i g_i(x^*) = 0$

These conditions characterize optimality for constrained optimization problems, especially under convexity assumptions.

7.10.2 Duality Gap and Sensitivity

- The **duality gap** is defined as the difference between the primal and dual objective values:

$$\text{Gap} = f(x^*) - q(\lambda^*, \mu^*)$$

- For convex problems with Slater's condition, this gap is zero.
- Sensitivity analysis uses Lagrange multipliers to understand how small changes in the problem data affect the optimal value. This is vital in economics and operations research.

7.10.3 Example: Nonlinear Dual Derivation

Consider:

$$\begin{aligned} \min_{x \in \mathbb{R}} \quad & x^2 \\ \text{s.t.} \quad & x - 1 \geq 0 \end{aligned} \Rightarrow \min_x x^2 \quad \text{s.t.} \quad -x + 1 \leq 0$$

Lagrangian:

$$L(x, \lambda) = x^2 + \lambda(-x + 1)$$

Dual function:

$$q(\lambda) = \inf_x (x^2 - \lambda x + \lambda) = \begin{cases} -\frac{\lambda^2}{4} + \lambda, & \text{no constraint on } x \\ \text{only valid if } \lambda \geq 0 \end{cases}$$

Dual problem:

$$\max_{\lambda \geq 0} \left(-\frac{\lambda^2}{4} + \lambda \right) \Rightarrow \lambda^* = 2, \quad q(\lambda^*) = 1$$

Primal solution: $x^* = 1, f(x^*) = 1$, confirming strong duality.

7.10.4 Extended Use Cases: Conic and Semidefinite Programming

- **Second-Order Cone Programming (SOCP):**

$$\min_x c^T x \quad \text{s.t.} \quad \|A_i x + b_i\|_2 \leq c_i^T x + d_i$$

Dual formulations exist and exhibit strong duality under conic Slater's condition.

- **Semidefinite Programming (SDP):**

$$\min_X \text{tr}(CX) \quad \text{s.t.} \quad \text{tr}(A_i X) = b_i, \quad X \succeq 0$$

SDP duality is used extensively in control, combinatorics, and robust optimization.

7.11 Sequential Convex Programming (SCP) and Successive Convex Approximation (SCA)

Sequential Convex Programming (SCP), or Successive Convex Approximation (SCA), addresses non-convex nonlinear programming (NLP) by solving a series of convex approximations. Each approximation is constructed around a current estimate and solved efficiently using convex optimization tools. This technique preserves mathematical rigor while remaining practically implementable in control, machine learning, and finance.

7.11.1 Problem Structure and Reformulation

We begin with the standard non-convex NLP:

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad g_i(x) \leq 0, \quad h_j(x) = 0$$

where:

- $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is non-convex but continuously differentiable
- g_i are non-convex inequality constraints
- h_j are nonlinear equality constraints

At each iteration k , we construct a surrogate convex program:

$$\min_x \tilde{f}^{(k)}(x) \quad \text{s.t.} \quad \tilde{g}_i^{(k)}(x) \leq 0, \quad \tilde{h}_j^{(k)}(x) = 0$$

where $\tilde{f}^{(k)}$, $\tilde{g}_i^{(k)}$, and $\tilde{h}_j^{(k)}$ are convex approximations of f , g_i , and h_j respectively.

7.11.2 Convexification Techniques

Let $x^{(k)}$ be the current iterate. Then:

- **Objective:** Quadratic model (proximal term):

$$\tilde{f}^{(k)}(x) = f(x^{(k)}) + \nabla f(x^{(k)})^T(x - x^{(k)}) + \frac{1}{2}(x - x^{(k)})^T H_k(x - x^{(k)})$$

where H_k is positive definite (e.g., $H_k = LI$ or quasi-Newton estimate)

- **Inequality Constraints:**

$$\tilde{g}_i^{(k)}(x) = g_i(x^{(k)}) + \nabla g_i(x^{(k)})^T(x - x^{(k)})$$

- **Equality Constraints:**

$$\tilde{h}_j^{(k)}(x) = h_j(x^{(k)}) + \nabla h_j(x^{(k)})^T(x - x^{(k)})$$

7.11.3 Algorithmic Framework

1. Initialize $x^{(0)}$ and select trust-region size Δ_0
2. At each iteration k :
 - Construct the surrogate convex subproblem
 - Solve to obtain $x^{(k+1)}$ using an interior-point or first-order method
 - Apply line search or trust-region adjustment
3. Stop when $\|x^{(k+1)} - x^{(k)}\| < \varepsilon$ and KKT residuals are small

7.11.4 Convergence Properties

SCP converges to a KKT point under the following conditions:

- The original problem satisfies constraint qualifications (e.g., LICQ, MFCQ)
- The surrogates are consistent (tangent at $x^{(k)}$) and convex upper bounds
- A merit function or filter strategy ensures sufficient descent

Merit Function Example:

$$\Phi(x) = f(x) + \rho \left(\sum_i [g_i(x)]_+ + \sum_j |h_j(x)| \right)$$

where $[\cdot]_+ = \max(\cdot, 0)$ and ρ is a penalty parameter.

7.11.5 Implementation Details

- Gradient and Jacobian evaluation must be efficient and accurate
- Hessian regularization (trust region or Levenberg-Marquardt style) prevents numerical instability
- Surrogates must remain feasible and convex — potentially requiring clipping or soft constraints

7.11.6 Mathematical Example: Cardinality-Constrained Portfolio Optimization

Non-convex objective:

$$\min_{x \in \mathbb{R}^n} x^T \Sigma x - \lambda \sum_{i=1}^n \log(x_i + \epsilon) \quad \text{s.t.} \quad \sum x_i = 1, \quad \|x\|_0 \leq K$$

SCP approximates $\log(x_i + \epsilon)$ via first-order model and relaxes $\|x\|_0$ using sorted ℓ_1 penalties or soft-thresholding with convex surrogates.

Convex Subproblem Example:

$$\min_x x^T \Sigma x + \mu \sum_i w_i^{(k)} x_i \quad \text{s.t.} \quad \sum x_i = 1, \quad x_i \geq 0$$

where $w_i^{(k)}$ is updated using a concave-convex procedure.

7.11.7 Applications

- Trajectory optimization
- Wireless network interference management
- Regularized logistic regression with non-convex priors
- Structured portfolio design with cardinality, turnover, or leverage constraints

7.11.8 Software Support

- CVXPY with DCP surrogate formulation
- CasADi and Acados for fast NLP surrogate compilation
- Pyomo + ipopt in loop

SCP is a foundational methodology in modern nonlinear optimization. By blending mathematical insight with tractable numerical routines, it offers a principled approach to solving complex decision problems in engineering, finance, and operations research.

7.12 Global Optimization for Nonconvex NLP

Global optimization methods seek globally optimal solutions to nonconvex nonlinear programs (NLPs), which may contain multiple local optima and disconnected feasible regions. Unlike local solvers, global techniques provide certificates of optimality by exhaustively exploring the feasible space or its relaxations. This section presents an in-depth theoretical and computational treatment of three core pillars: Branch-and-Bound (B&B), Convex Relaxations (e.g., McCormick), and global solvers. Global optimization for NLPs combines rigorous bounding, mathematical relaxations, and adaptive tree search to solve nonconvex programs reliably. Its principles underpin many real-world applications in finance, engineering design, operations research, and control, where global solutions are essential.

7.12.1 Branch-and-Bound for Nonconvex NLPs

Branch-and-bound recursively partitions the decision space and constructs valid lower bounds on the objective value within each partition. The process continues until the global optimum is identified or approximated to within a specified tolerance.

7.12.1.1 Mathematical Framework Consider the general NLP:

$$\begin{aligned} \min_{x \in X} \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

where f , g_i , and h_j may be nonconvex, and X is a box domain.

At each node of the B&B tree:

- Solve a convex relaxation to obtain a lower bound L_k
- Apply a local solver (e.g., SQP, IPM) to find an upper bound U_k
- Prune the node if $L_k > U^*$ (current global best)
- Branch by bisecting variable bounds or relaxing integer constraints

Termination condition: $\max_k (U_k - L_k) < \varepsilon$.

7.12.1.2 Bounding and Feasibility To construct valid bounds, subproblems are convexified using underestimators. Feasibility can be managed via constraint propagation, interval arithmetic, or convex outer approximations of feasible sets.

7.12.1.3 Example: Nonconvex Quadratic Program

$$\begin{aligned} \min_{x \in [-2, 2]^2} \quad & x_1^2 + x_2^2 - 3x_1x_2 + \sin(x_1) \\ \text{s.t.} \quad & x_1x_2 \leq 1 \end{aligned}$$

The nonconvex term x_1x_2 is relaxed via McCormick envelopes; $\sin(x_1)$ can be bounded with piecewise linear convex approximations.

7.12.2 Convex Relaxations and Underestimation

Convex relaxations approximate nonconvex functions from below to ensure valid lower bounds. These are crucial in bounding subproblems and guiding pruning decisions.

7.12.2.1 McCormick Envelopes Given $z = xy$ with $x \in [x^L, x^U]$, $y \in [y^L, y^U]$, define the relaxation:

$$\begin{aligned} z &\geq x^L y + x y^L - x^L y^L \\ z &\geq x^U y + x y^U - x^U y^U \\ z &\leq x^L y + x y^U - x^L y^U \\ z &\leq x^U y + x y^L - x^U y^L \end{aligned}$$

These envelopes form a tight polyhedral relaxation of bilinear terms. For multilinear or trigonometric terms, these can be composed or approximated via piecewise convex envelopes.

7.12.2.2 Factorable Function Relaxations A function is *factorable* if it can be constructed using a finite composition of univariate and bilinear functions. Relaxation proceeds recursively:

- Univariate terms (e.g., $\exp(x)$, $\log(x)$) are convexified over bounded intervals
- Bilinear products are relaxed using McCormick bounds
- Composite rules propagate convex/concave envelopes through arithmetic trees

7.12.2.3 Piecewise Convex Relaxations Partitioning the domain into regions and convexifying within each yields tighter bounds at increased cost. For example, a bilinear term on $[-1, 1] \times [-1, 1]$ can be split into four quadrants and relaxed per region.

7.12.3 Deterministic vs. Stochastic Global Solvers

7.12.3.1 Deterministic Solvers These solvers provide global optimality guarantees using relaxation-based B&B or spatial branch-and-reduce frameworks.

- **BARON:** Combines LP/NLP relaxations, convex envelopes, and local solvers
- **Couenne:** Extends IPOPT with global NLP via constraint propagation, symbolic reformulations, and McCormick tightening
- **SCIP, ANTIGONE:** Handle discrete-continuous MINLPs with linear and nonlinear relaxations

7.12.3.2 Stochastic and Metaheuristic Methods These methods use randomization and population-based search:

- **Simulated Annealing (SA):** Perturbation and temperature-based acceptance criteria
- **Genetic Algorithms (GA):** Crossover and mutation over candidate populations
- **Particle Swarm Optimization (PSO):** Velocity-updated search points
- **Bayesian Optimization:** Gaussian process surrogate + acquisition maximization

These methods do not offer certificates of global optimality but are useful in high-dimensional or black-box settings.

7.12.4 Computational Trade-offs and Enhancements

- **Tightness vs. complexity:** Stronger relaxations lead to smaller trees but greater per-node solve time
- **Bound tightening:** Interval contraction and reduced domains speed up convergence
- **Hybrid approaches:** Use local NLP solvers (e.g., IPOPT, SNOPT) to update incumbent bounds
- **Symbolic preprocessing:** Algebraic simplification and structure detection improve relaxations

References

- [1] Wu, M. L. (2025). *Optimization Repository*. GitHub. Available at: <https://github.com/Zontafor/optimization>